RACING

R-based Attractor Complexity INdex for Gait

User Guide

Complete Function Reference and Pipeline Documentation

Version 1.1 — March 2026
ORD 2025 Project
HE-Arc Santé Neuchâtel

Philippe Terrier
HE-Arc Santé, HES-SO University of Applied Sciences and Arts Western Switzerland

Table of Contents

Preface

This User Guide provides comprehensive documentation for the RACING (R-based Attractor Complexity INdex for Gait) software package. RACING is an open-source R implementation of validated MATLAB algorithms for linear and nonlinear gait analysis using trunk-worn accelerometer data. The package was developed as part of the ORD 2025 (Open Research Data) project at HE-Arc Santé Neuchâtel, with funding from the HES-SO University of Applied Sciences and Arts Western Switzerland.
The algorithms implemented in RACING were originally developed and validated in the ACIER study (Attractor Complexity Index Empirical Rationalization, 2021–2024), which analyzed gait quality in older adults using triaxial accelerometer data from a lower-back sensor. The MATLAB implementations were published alongside two peer-reviewed articles in Scientific Reports and Sensors. The RACING project makes these algorithms freely available to the international research community by porting them to the open-source R language, following FAIR (Findable, Accessible, Interoperable, Reusable) principles.

How to Use This Guide

This guide is organized following the data processing pipeline, from raw accelerometer data to final gait quality metrics. Each chapter corresponds to one or more R scripts and provides the scientific background, function signatures, parameter descriptions, usage examples, error handling, and integration notes.
Chapter 1 introduces the scientific context and project background. Chapter 2 describes the pipeline architecture and processing order. Chapters 3–12 document individual analysis scripts in pipeline order. The Appendices provide a quick reference card, glossary of terms, and complete bibliography.

Conventions

Throughout this guide, R code is displayed in monospaced font. Function names are written as `function_name()`. File names are shown as `script_name.R`. Default parameter values appear in square brackets, e.g., [256]. Cross-references to other chapters are indicated by chapter number.

Software Requirements

RACING requires R version 4.0 or later. The following packages are needed: gsignal (MATLAB-compatible signal resampling), RANN (fast nearest-neighbor search via KD-trees, strongly recommended), data.table (fast CSV import, optional), readxl (reading Excel configuration files), and openxlsx (writing Excel output files).

```
install.packages(c("gsignal", "RANN", "data.table", "readxl", "openxlsx"))
```

The base R package parallel is used for across-axis parallelization of the nonlinear analysis and requires no installation.

1. Introduction

1.1 Scientific Context

Falls in older adults represent a major public health challenge. Approximately one-third of adults over 65 experience at least one fall per year, often with serious consequences including fractures, loss of autonomy, and increased mortality. Early detection of gait instability could enable preventive interventions before falls occur.
Nonlinear dynamical analysis of trunk acceleration during walking offers a promising approach to quantifying gait quality beyond traditional clinical measures. By reconstructing the phase space attractor from a single accelerometer signal, researchers can assess both local dynamic stability (sensitivity to perturbations within a gait cycle) and attractor complexity (the structure of stride-to-stride fluctuations). These measures have been shown to distinguish between fallers and non-fallers, between younger and older adults, and between different walking conditions.

1.2 The ACIER Study

The ACIER study (Attractor Complexity Index Empirical Rationalization) was conducted at HE-Arc Santé Neuchâtel between 2021 and 2024. It collected triaxial accelerometer data from 102 participants (older and younger adults) wearing a Physilog sensor on the lower back at 256 Hz. Participants walked under multiple conditions: normal indoor, metronome-paced indoor, and normal outdoor. The study produced two publications: Piergiovanni & Terrier (2024) in Scientific Reports on effects of metronome walking on long-term attractor divergence, and Piergiovanni & Terrier (2024) in Sensors on validity of linear and nonlinear measures of gait variability.
The raw accelerometer data are publicly available on Zenodo (DOI: 10.5281/zenodo.10148824).

1.3 The RACING Project

RACING (R-based Attractor Complexity INdex for Gait) is an 8-month ORD 2025 project that ports the validated MATLAB algorithms from the ACIER study to R, making them open-source and accessible to researchers worldwide. The project follows FAIR principles: the source code is hosted on GitHub, the software is described in a SoftwareX article [pending pubilication], and a CodeOcean reproducibility capsule provides a citable, executable demonstration.
The R implementation was developed with a validation-first approach: each function was verified against the original MATLAB code line-by-line, and cross-platform numerical agreement was confirmed on the full ACIER dataset. The pipeline achieves exact agreement for linear metrics (step frequency, RMS, ACF regularity) and near-perfect agreement for nonlinear metrics, with a documented systematic offset of approximately 0.008 in ACI values attributable to differences in polyphase resampling filter implementations between MATLAB and R.

1.4 Gait Quality Metrics

RACING computes eight categories of gait metrics from a single lower-back accelerometer recording: step frequency (SF), RMS movement intensity (RMS_norm), RMS lateral ratio (RMS_ratio), step regularity (step_reg), stride regularity (stride_reg), Average Mutual Information (AMI), Local Dynamic Stability (LDS), and Attractor Complexity Index (ACI).

2. Pipeline Architecture

2.1 Processing Order

The RACING pipeline processes each accelerometer file through a fixed sequence of operations. The processing order is critical and must not be altered, as several steps depend on outputs from previous steps. The sequence is: (0) compute vector norm from raw signals, (1) tilt correction, (2) step frequency detection on corrected vertical, (3) SF rounding to 0.1 Hz, (4) signal truncation, (5) signal resampling, (6) RMS computation, (7) ACF gait regularity, (8) AMI for 4 axes, (9) Rosenstein divergence for 4 axes, and (10) LDS/ACI fitting for 4 axes.

2.2 Dual Signal Architecture

A key design principle is the production of two distinct signal versions from the preprocessing stage. Truncated signals are cut to a standardized number of steps at the original sampling rate (e.g., 256 Hz). These preserve actual sampling rate and are used for linear metrics. Resampled signals are further processed to a fixed length (default: 18,750 samples = 250 steps at 75 samples/step). This standardization is required for valid phase space reconstruction and inter-subject comparison of nonlinear metrics.

2.3 Script Layers

The 13 R scripts are organized in three functional layers. Layer 1 (standalone metrics): `tilt_correction.R`, `step_frequency_detection.R`, `acf_gait_regularity.R`, `ami.R`, `rosenstein_divergence.R`, `fit_divergence_exponents.R`, `compute_rms.R`, `resample_signal.R`. Layer 2 (preprocessing orchestration): `truncate_resample.R`. Layer 3 (batch processing): `read_config.R`, `import_csv_lowback.R`, `run_gait_analysis_batch.R`, and `write_results.R`.

2.4 Configuration System

All analysis parameters are stored in a RACING configuration Excel file (RACING_config.xlsx). This centralized design ensures reproducibility: the same configuration file applied to the same data will always produce identical results.

3. tilt_correction.R

Tilt Correction Algorithm

1. Purpose

`tilt_correction.R` implements the Moe-Nilssen (1998) tilt correction algorithm, which transforms triaxial accelerometer data from an arbitrarily tilted sensor frame into a horizontal-vertical earth frame, removing the effect of gravity. The script contains three exported functions: `tilt_correction()` (main algorithm), `tilt_correction_batch()` (data frame wrapper), and `validate_tilt_correction()` (post-hoc quality check).

The corrected signals feed into `truncate_resample.R` where step frequency detection is performed on the corrected vertical axis. The correction ensures that the gravity component is properly removed and that each axis reflects only dynamic accelerations in the anatomical horizontal-vertical reference frame.

2. Scientific Background

When a triaxial accelerometer is attached to the lower back during walking, it is rarely perfectly aligned with the anatomical axes. Even small sensor tilts cause gravity (1g) to project onto the horizontal axes, creating a static offset that contaminates the AP and ML acceleration signals. For a typical forward tilt of 10 degrees, the AP axis registers a static 0.17g offset (= sin(10 deg)).

The Moe-Nilssen algorithm exploits the fact that over a complete walking trial, the mean dynamic acceleration on each axis approaches zero (cyclical movement). Therefore, the measured mean acceleration equals the gravity component, which is proportional to sin(theta) for the tilted axis. Two sequential 2D rotations then transform the data back to the earth frame.

2.1 The Algorithm (Equations A1-A4)

Step 1: Estimate tilt angles from mean accelerations. theta_a = arcsin(mean(a_ap)) for the sagittal plane tilt, and theta_m = arcsin(mean(a_ml)) for the frontal plane tilt.

Step 2: First rotation corrects AP tilt in the sagittal plane, producing a corrected horizontal AP acceleration (Eq. A1) and a provisional vertical acceleration (Eq. A2). Step 3: Second rotation corrects ML tilt in the frontal plane using the provisional vertical from Step 2, producing the corrected ML (Eq. A3) and the final vertical with gravity removed (Eq. A4).

Rotation equations:

Eq.	Formula	Meaning
A1	`a_A = a_ap * cos(theta_a) - a_v * sin(theta_a)`	Corrected AP (horizontal)
A2	`a_v' = a_ap * sin(theta_a) + a_v * cos(theta_a)`	Provisional vertical
A3	`a_M = a_ml * cos(theta_m) - a_v' * sin(theta_m)`	Corrected ML (horizontal)
A4	`a_V = a_ml * sin(theta_m) + a_v' * cos(theta_m) - 1`	Final vertical (gravity removed)

The -1 in Equation A4 removes the static 1g gravity component so that the corrected vertical acceleration has a mean near zero, like the horizontal axes.

3. Function Reference

3.1 tilt_correction()

Main analysis function. Applies two sequential 2D rotations and removes gravity.

Parameters:

Parameter	Type	Default	Description
acc_ap	numeric	(required)	AP acceleration vector in g units
acc_v	numeric	(required)	Vertical acceleration vector in g units
acc_ml	numeric	(required)	ML acceleration vector in g units
auto_orient	logical	TRUE	Auto-detect and correct inverted sensor mounting
return_diagnostics	logical	FALSE	Return tilt angles, orientation, quality check
verbose	logical	FALSE	Print diagnostic messages to console

Output fields (return_diagnostics = FALSE):

Field	Description
ap	Corrected AP acceleration (mean ~ 0)
v	Corrected vertical acceleration, gravity removed (mean ~ 0)
ml	Corrected ML acceleration (mean ~ 0)

Additional output fields (return_diagnostics = TRUE):

Field	Description
theta_ap / theta_ml	Estimated tilt angles in radians
theta_ap_deg / theta_ml_deg	Estimated tilt angles in degrees
orientation_detected	'upward' or 'downward' (sensor mounting)
orientation_corrected	TRUE if vertical signal was flipped
mean_input / mean_output	Named vectors of axis means before/after correction
quality_check	TRUE if all output means are near zero

3.2 tilt_correction_batch()

Convenience wrapper for data frames and matrices. Extracts AP, V, and ML columns by name or index, applies `tilt_correction()`, and returns a data frame with columns `ap_corrected`, `v_corrected`, and `ml_corrected`.

3.3 validate_tilt_correction()

Post-hoc quality check on corrected data. Verifies that output means are within tolerance (AP/ML: 0.05g, V: 0.10g by default) and that estimated tilt angles are within a reasonable range (default: 30 degrees). The V tolerance is wider because the zero-mean-dynamic-acceleration assumption is more sensitive to short signals and incomplete final strides.

4. Usage Example

```
source('tilt_correction.R')

# Load raw triaxial data (x=AP, y=V, z=ML in g units)
result <- tilt_correction(
    acc_ap = raw_x,
    acc_v  = raw_y,
    acc_ml = raw_z,
    return_diagnostics = TRUE,
    verbose = TRUE
)

# Use corrected signals
corrected_x <- result$ap     # AP, mean ~ 0
corrected_y <- result$v      # V, gravity removed
```

```
corrected_z <- result$ml    # ML, mean ~ 0

# Check quality
validation <- validate_tilt_correction(result)
if (!validation$valid) warning(validation$messages)
```

5. Pipeline Integration

In the RACING batch pipeline, `tilt_correction()` is called by `truncate_resample.R` as the first processing step. The axis mapping follows the ACIER convention: acc_x = AP, acc_y = V, acc_z = ML. Corrected signals replace the raw ones in-place, after which step frequency detection and truncation proceed on the corrected vertical axis.

The vector norm is computed before tilt correction (as a rotation, the correction preserves the norm mathematically). This matches the MATLAB processing order. Step frequency must be detected after tilt correction because the rotation redistributes energy between axes, which can shift the FFT peak by up to 1 bin.

Relevant batch config parameters:

Config Key	Default	Description
`apply_tilt`	TRUE	Enable/disable Moe-Nilssen tilt correction
`auto_orient`	TRUE	Auto-detect inverted sensor orientation

6. Error Handling

Condition	Response	Rationale
Non-numeric inputs	`stop()`	Type validation
Vectors different length	`stop()`	Length check
n < 10 samples	`warning()`	Angle estimate unreliable
NA values in input	`warning()`	Propagate to output with na.rm for means
|sin(theta)| > 1.0	`clamp to 0.999`	Noise can push mean beyond physical limit
|sin(theta)| > 1.1	`warning()`	Likely wrong units or sensor malfunction
Output means not near 0	`quality_check=FALSE`	Flagged in diagnostics and validation
Extreme tilt > 30 deg	`warning (batch)`	Unusual sensor placement

7. Methodological Notes

Orientation detection. The R implementation adds automatic orientation detection not present in the original paper. When mean(V) < -0.1g, the vertical signal is flipped before applying the rotation equations. The threshold of -0.1g reliably distinguishes normal mounting (mean V ~ +0.98g) from inverted mounting (mean V ~ -0.98g) while avoiding false triggers from noise.

Quality check tolerances. The AP and ML axes use a 0.05g tolerance because the rotation equations directly cancel the gravity component on these axes; larger residuals indicate real problems. The V axis uses a wider 0.10g tolerance because it depends on the zero-mean-dynamic-acceleration assumption, which is more sensitive to short signals, incomplete final strides, or mildly asymmetric gait.

Rotation order. The AP correction must precede the ML correction because the second rotation uses the provisional vertical from the first. Reversing the order would give different

(incorrect) results. This sequential rotation approach assumes small angles; for tilts up to ~28 degrees (typical gait range), the approximation is excellent as validated in the original paper.

Limitations. The algorithm cannot correct for yaw rotation (rotation about the vertical axis). This is not a problem for controlled laboratory settings with standardized sensor placement, but can be an issue for free-living monitoring where the sensor may rotate during extended wearing periods.

8. References

Moe-Nilssen, R. (1998). A new method for evaluating motor control in gait under real-life environmental conditions. Part 1: The instrument. Clinical Biomechanics, 13(4-5), 320-327. https://doi.org/10.1016/s0268-0033(98)00089-8

Piergiovanni, S. & Terrier, P. (2024). Validity of linear and nonlinear measures of gait variability to characterize aging gait with a single lower back accelerometer. Sensors, 24(23), 7427. https://doi.org/10.3390/s24237427

4. step_frequency_detection.R

Step Frequency Detection

1. Purpose

`step_frequency_detection.R` estimates step frequency from vertical trunk acceleration using FFT power spectral density analysis. The script contains four functions: `step_frequency_fft()` (main analysis), `step_frequency()` (convenience wrapper), `plot_step_frequency()` (diagnostic visualization), and `test_step_frequency()` (self-test with synthetic signal).

The detected step frequency serves two downstream purposes: (1) scientific reporting of cadence, and (2) computing the truncation index in truncate_resample.R to extract a standardized number of steps from each walking trial.

2. Scientific Background

Step frequency is the rate of stepping during walking, typically 1.5-2.3 Hz (90-138 steps/min) for adults. In the ACIER study (Piergiovanni & Terrier, 2024), step frequency was derived from the FFT of the tilt-corrected vertical acceleration signal, where the dominant spectral peak within the walking frequency range corresponds to the step rate.

The R implementation adds parabolic (quadratic) interpolation on the log-magnitude spectrum to achieve sub-bin frequency precision. This enhancement is described by Gasior and Gonzalez (2004) and provides robustness against the +/-1 FFT bin shifts (~0.004 Hz) that can arise from floating-point differences in tilt correction between MATLAB and R.

2.1 The SF Rounding Mechanism

To ensure MATLAB/R convergence for truncation, both implementations round SF to 0.1 Hz before computing the truncation index. The precise SF is preserved separately for scientific reporting. This design prioritizes having a coherent number of steps (identical truncation lengths) over maximum frequency precision, since the truncation index is highly sensitive to small SF differences: a 0.004 Hz shift can change the truncation by ~50 samples.

```
MATLAB:  SF_trunc = round(SF, 1);  stopind = floor((1/SF_trunc)*fs*sn);
R:       step_freq_hz <- round(step_freq_hz, 1)
         truncation_index <- floor(sampling_freq / step_freq_hz * target_steps)
```

3. Function Reference

3.1 step_frequency_fft()

Main analysis function. Computes step frequency via FFT with parabolic interpolation.

Parameters:

Parameter	Type	Default	Description
`acceleration_vertical`	numeric	`(required)`	Vertical acceleration vector in g units
`sampling_freq`	numeric	`256`	Sampling frequency in Hz
`freq_range`	numeric[2]	`c(1.4, 3.0)`	Min/max frequency search range (Hz). Pipeline may override to c(1.2, 3.0)
`detrend`	logical	`TRUE`	Remove signal mean before FFT (matches MATLAB y-mean(y))
`zero_pad`	logical	`FALSE`	Zero-pad to next power of 2. Unnecessary with interpolation
`return_spectrum`	logical	`FALSE`	Include PSD and frequency vectors in output (for plotting)

Output fields:

Field	Description
`step_freq_hz`	Interpolated step frequency (Hz) -- primary output
`step_freq_bpm`	Step frequency in beats per minute
`peak_bin_freq`	Raw FFT bin frequency before interpolation (diagnostic)
`peak_bin_index`	1-indexed FFT bin of discrete peak
`interp_delta`	Interpolation offset in fractional bins (-0.5 to +0.5)
`peak_power`	PSD value at the peak bin
`freq_resolution`	FFT bin width df = fs/N (Hz)
`method`	Always 'FFT_parabolic'
`quality`	Sub-list: interp_valid, warnings
`psd / frequencies`	Full spectrum vectors (when return_spectrum = TRUE)

Algorithm:

1. Detrend: subtract mean from input signal. 2. Compute FFT (no windowing, matching MATLAB). 3. Compute PSD as |X(f)|^2 / N for positive frequencies. 4. Find the maximum-amplitude bin within freq_range using which.max. This is equivalent to MATLAB's first-peak selection because within [1.4, 3.0] Hz the step frequency is always the dominant peak (stride frequency is below 1.4 Hz; harmonics are above 3.0 Hz). 5. Apply parabolic interpolation on log(|X|) using the peak bin and its two neighbors: delta = 0.5*(alpha-gamma)/(alpha-2*beta+gamma). 6. Compute interpolated frequency: f = (k-1+delta)*df where k is 1-indexed.

3.2 plot_step_frequency()

Diagnostic two-panel visualization. Requires step_frequency_fft() output with return_spectrum = TRUE.

Parameters:

Parameter	Type	Default	Description
`signal`	numeric	`(required)`	Original acceleration signal
`result`	list	`(required)`	Output from step_frequency_fft(return_spectrum=TRUE)
`sampling_freq`	numeric	`256`	Sampling frequency in Hz
`xlim_psd`	numeric[2]	`c(0, 5)`	X-axis limits for PSD panel

Panel 1 (top): Time series in steelblue with detected frequency in legend. Panel 2 (bottom): PSD curve in dark blue with green-shaded search range, gray open circle on the discrete bin peak, red filled circle and dashed vertical line at the interpolated frequency, and a label showing bin frequency, interpolated frequency, and delta offset. Y-axis is scaled to the search range maximum to prevent harmonic domination. Graphics parameters are saved/restored via on.exit(par(old_par)) with RStudio-compatible margins.

3.3 step_frequency()

Convenience wrapper that forwards all arguments to `step_frequency_fft()`. Allows calling code to use the shorter name without losing access to any parameters.

3.4 test_step_frequency()

Self-test using a synthetic gait-like signal (sinusoid at 1.8765 Hz + stride harmonic + noise). Reports bin error vs interpolation error, improvement ratio, and truncation equivalence. Includes a bin boundary robustness test that generates a second signal with a tiny phase shift to verify that interpolation converges even when the discrete peak shifts by one bin. Both raw and pipeline-rounded (0.1 Hz) truncation indices are compared.

4. Usage Example

```
source('step_frequency_detection.R')
source('tilt_correction.R')

# After tilt correction on vertical acceleration:
sf <- step_frequency_fft(acc_y_corrected, sampling_freq = 256,
                         return_spectrum = TRUE)
cat(sprintf('SF = %.4f Hz (%.1f bpm)\n',
    sf$step_freq_hz, sf$step_freq_bpm))

# Pipeline truncation uses rounded value:
sf_rounded <- round(sf$step_freq_hz, 1)
trunc_idx <- floor(256 / sf_rounded * 250)

# Diagnostic plot:
plot_step_frequency(acc_y_corrected, sf)
```

5. Pipeline Integration

In the RACING batch pipeline, `step_frequency_fft()` is called in two contexts. First, `run_gait_analysis_batch.R` calls it externally for quality warning collection. Second, `truncate_resample.R` calls it internally on the tilt-corrected vertical signal. When tilt correction is active (default), the internal call always takes precedence, overriding any externally-provided step_freq_hz value. This ensures SF is detected on the same corrected signal used for all downstream processing.

The truncation formula is algebraically identical to MATLAB: floor(sampling_freq / round(SF, 1) * target_steps). The round(SF, 1) step ensures that tiny FFT bin differences between MATLAB and R (caused by floating-point tilt correction differences) never propagate into truncation length mismatches.

Relevant batch config parameters:

Config Key	Default	Description
`sf_freq_min`	1.2	Lower bound of SF search range (Hz)
`sf_freq_max`	3.0	Upper bound of SF search range (Hz)
`target_steps`	250	Number of steps to extract (250 indoor, 500 outdoor)
`samples_per_step`	75	Samples per step after resampling

6. Error Handling

Condition	Response	Rationale
Signal < 100 samples	`stop()`	Input too short for meaningful FFT
Signal < 4 seconds	`warning()`	Reduced frequency resolution
< 3 bins in freq_range	`stop()`	Range too narrow for interpolation
Peak at spectrum edge	`warning`	Interpolation skipped, raw bin used
Zero magnitude neighbor	`warning`	log(0) would fail; interpolation skipped
Non-concave peak shape	`warning`	Flat spectral plateau; interpolation skipped
|delta| > 0.5	`warning`	Clamped to +/-0.5 with warning

7. Methodological Notes

which.max vs first-peak. MATLAB's SF_det.m uses peakn-based peak detection selecting the first (lowest frequency) peak in the search range. The R script uses which.max (maximum amplitude). These give identical results within [1.4, 3.0] Hz because stride frequency (~0.9-1.2 Hz) falls below the lower bound and step harmonics (~3.6+ Hz) fall above the upper bound.

Log-magnitude interpolation. For signals analyzed with a rectangular window (no windowing), log-magnitude parabolic interpolation provides zero bias, making it the optimal choice per Gasior and Gonzalez (2004). The formula delta = 0.5*(alpha-gamma)/(alpha-2*beta+gamma) where alpha, beta, gamma are log-magnitudes of the bins flanking the peak.

Rounding for truncation convergence. With typical gait SF around 1.8-2.0 Hz and FFT bin width ~0.004 Hz, the maximum MATLAB-R bin discrepancy (~0.004 Hz) is well within the +/- 0.05 Hz rounding tolerance of round(SF, 1). This ensures both implementations always arrive at the same truncation index, which is important for downstream resampling and metric comparability.

PSD normalization. The script computes PSD as |X(f)|^2 / N (standard periodogram). Peak location is identical whether using magnitude or PSD since squaring preserves the maximum position.

8. References

Gasior, M. & Gonzalez, J.L. (2004). Improving FFT frequency measurement resolution by parabolic and Gaussian interpolation. AIP Conf Proc 732(1), 276-285.

Piergiovanni, S. & Terrier, P. (2024). Effects of metronome walking on long-term attractor divergence and correlation structure of gait. Scientific Reports, 14, 15784. https://doi.org/10.1038/s41598-024-65662-5

Piergiovanni, S. & Terrier, P. (2024). Validity of linear and nonlinear measures of gait variability to characterize aging gait with a single lower back accelerometer. Sensors, 24(23), 7427. https://doi.org/10.3390/s24237427

Smith, J.O. (2011). Spectral Audio Signal Processing. W3K Publishing. Chapter: Quadratic Interpolation of Spectral Peaks.

5. truncate_resample.R

Gait signal Signal Truncation and Resampling Pipeline

1. Purpose

`truncate_resample.R` is the central preprocessing orchestrator of the RACING pipeline. It transforms raw triaxial accelerometer signals into two analysis-ready signal sets: (1) truncated signals at the original sampling rate for linear metrics (RMS, step frequency, ACF regularity), and (2) resampled signals at a standardized length for nonlinear dynamics analysis (AMI, LDS, ACI). The function integrates three sub-modules -- tilt correction, step frequency detection, and bandlimited resampling -- into a single call that replicates the exact processing order of the MATLAB ACIER pipeline.

2. Scientific Background

2.1 Why Two Output Versions?

Linear gait metrics (RMS movement intensity, step frequency, ACF-based step/stride regularity) are computed at the original sampling rate. Nonlinear metrics (AMI, LDS, ACI) operate on reconstructed phase space attractors and require standardized time scales: every subject's signal must have exactly the same number of samples per step for valid inter-subject comparison. The resampling step achieves this standardization by transforming all signals to exactly 75 samples per step, regardless of the subject's actual cadence.

2.2 Processing Order

The MATLAB ACIER pipeline performs operations in a specific order: (1) compute vector norm from raw signals, (2) apply tilt correction, (3) detect step frequency on the corrected vertical acceleration, (4) round SF to 0.1 Hz, (5) truncate, (6) resample.

2.3 Tilt Correction

The Moe-Nilssen (1998) tilt correction algorithm compensates for sensor misalignment by using the principle that gravity should only be detected on the vertical axis of a perfectly aligned sensor. The algorithm estimates tilt angles from the mean acceleration on each axis and applies a rotation matrix to redistribute the gravitational component. The function also supports automatic detection of inverted sensor orientation (upside-down mounting).

2.4 Step Frequency Rounding

Both MATLAB and R round the detected step frequency to 0.1 Hz before computing the truncation index. This was introduced in the RACING project to ensure MATLAB/R convergence: the parabolic interpolation in step frequency detection can produce slightly different sub-bin precision across platforms, but rounding to 0.1 Hz eliminates this source of divergence while preserving the precise value in metadata for reporting.

2.5 Resampling and Problematic Ratios

Resampling uses bandlimited sinc interpolation to change the signal length while preserving frequency content below the Nyquist limit. When the resampling ratio (target/source) is very close to 1.0, the polyphase filter becomes numerically unstable (too many phases). The R implementation detects these cases automatically and applies two-stage resampling, eliminating the need for subject-specific workarounds.

3. Processing Pipeline

The function executes the following steps in order:

Step	Operation	Method	Notes
Step 0	Compute Norm	sqrt(x^2+y^2+z^2) - 1	Raw signals (before correction)
Step 1	Tilt Correction	Moe-Nilssen 1998 rotation matrix	Optional; auto-orient supported
Step 2	SF Detection	FFT on corrected vertical acc	Always internal when tilt ON
Step 3	SF Rounding	round(SF, 1)	Ensures M/R convergence
Step 4	Truncation Index	floor(fs / SF * N_steps)	Short signal handling included
Step 5	Truncate	signal[1:truncation_index]	All axes + pre-computed norm
Step 6	Target Length	N_steps x samples_per_step	Default: 250 x 75 = 18750
Step 7	Resample	Bandlimited sinc interpolation	Auto two-stage for ratio ~1
Step 8	Output	S3 gait_preprocessed object	truncated + resampled + metadata

4. Function Reference

4.1 truncate_resample()

Main preprocessing function. Applies tilt correction, detects step frequency, truncates, and resamples triaxial accelerometer signals.

Parameter	Type	Default	Description
`acc_x`	numeric	`(required)`	Anteroposterior (AP) acceleration in g units
`acc_y`	numeric	`(required)`	Vertical (V) acceleration in g units
`acc_z`	numeric	`(required)`	Mediolateral (ML) acceleration in g units
`sampling_freq`	numeric	`256`	Sampling frequency in Hz
`target_steps`	integer	`250`	Number of steps to extract
`target_samples`	integer/NULL	`NULL`	Target resampled length; NULL = target_steps x samples_per_step
`samples_per_step`	integer	`75`	Samples per step after resampling (ACIER standard)
`apply_tilt_correction`	logical	`TRUE`	Apply Moe-Nilssen tilt correction
`auto_orient`	logical	`TRUE`	Auto-detect inverted sensor orientation
`step_freq_hz`	numeric/NULL	`NULL`	Pre-computed SF; IGNORED when tilt correction is ON
`freq_range`	numeric(2)	`c(1.2, 3.0)`	Min/max Hz for SF detection
`compute_norm`	logical	`TRUE`	Include vector norm in outputs
`norm_subtract_gravity`	logical	`TRUE`	Subtract 1g from norm
`resampling_method`	character	`"auto"`	"auto", "direct", or "two_stage"
`verbose`	logical	`TRUE`	Print progress messages
`log_file`	character/NULL	`NULL`	Path to debug log file for resampling

4.2 Return Value

The function returns an S3 object of class `gait_preprocessed` with custom print() and summary() methods. The object is a list with the following components:

Component	Type	Contents

truncated	list	Truncated signals at original rate: x, y, z, norm
resampled	list	Resampled signals at standardized length: x, y, z, norm
metadata	list	Processing parameters: step_freq_hz, step_freq_bpm, target_steps, actual_steps, truncated_length, resampled_length, samples_per_step, sampling_freq, resampling_ratio, resampling_method, signal_truncated, signal_too_short, tilt_correction_applied
tilt_correction	list/NULL	Tilt diagnostics: theta_ap_deg, theta_ml_deg, orientation_detected, orientation_corrected
quality	list	Quality indicators: tilt_correction_valid, warnings (character vector)

4.3 print.gait_preprocessed()

S3 print method that displays a formatted summary of preprocessing results: tilt correction angles, step frequency, truncated/resampled signal lengths, resampling method, and any warnings. Called automatically when the result object is printed to the console.

4.4 summary.gait_preprocessed()

S3 summary method that displays per-component statistics (mean, SD, range) for both truncated and resampled signals, plus key metadata. Useful for quick quality assessment.

5. Usage Examples

5.1 Standard ACIER Analysis (NC condition, 250 steps)

```
source("tilt_correction.R")
source("step_frequency_detection.R")
source("resample_signal.R")
source("truncate_resample.R")

data <- read.csv("subject_001_NC.csv")
result <- truncate_resample(
  acc_x = data$acc_x, acc_y = data$acc_y, acc_z = data$acc_z,
  sampling_freq = 256, target_steps = 250,
  samples_per_step = 75, verbose = TRUE
)
print(result)  # Formatted summary
```

5.2 Accessing Outputs for Downstream Analysis

```
# For RMS and ACF regularity (truncated, original rate)
rms <- compute_rms(az = result$truncated$z, norm_signal = result$truncated$norm)
reg <- compute_gait_regularity(acc_norm = result$truncated$norm, ...)

# For AMI, LDS, ACI (resampled, standardized)
ami_result <- ami(result$resampled$norm, n_bins = 64, n_lags = 100)
div <- rosenstein_divergence(result$resampled$norm, m=5, tau=ami_result$optimal_lag, ...)
```

5.3 Pre-corrected Data (Skip Tilt Correction)

```
result <- truncate_resample(
  acc_x = corrected_x, acc_y = corrected_y, acc_z = corrected_z,
  apply_tilt_correction = FALSE,
  step_freq_hz = 1.85  # Used directly (not overridden)
)
```

5.4 Quality Checking

```
if (result$metadata$signal_too_short)
  warning(sprintf("Only %.1f steps (target: %d)",
```

```
      result$metadata$actual_steps, result$metadata$target_steps))

if (!result$quality$tilt_correction_valid)
  warning("Tilt correction quality check failed")

if (length(result$quality$warnings) > 0)
  cat(paste(result$quality$warnings, collapse = "\n"))
```

6. Dual Output Architecture

The function produces two complete signal sets with distinct purposes:

Output	Length	Sample rate	Used for	Rationale
`truncated`	floor(fs/SF*N)	Original (256 Hz)	RMS, SF, ACF	Actual sampling rate
`resampled`	N*75 = 18750	Standardized	AMI, LDS, ACI	Uniform samples/step for phase space

Both output sets include x (AP), y (V), z (ML), and norm (vector magnitude). The norm is always computed from raw signals before tilt correction, ensuring orientation-invariant measurement.

7. Error Handling and Warnings

Condition	Response	Details
Non-numeric inputs	stop()	All 3 axes checked
Unequal vector lengths	stop()	x, y, z must match
Signal < 256 samples	stop()	Minimum 1 second required
Invalid parameters	stop()	sampling_freq, target_steps, samples_per_step
NA values in signals	warning()	Stored in quality$warnings
Signal too short for N steps	warning()	Uses all samples; actual_steps reported
Tilt correction quality fail	warning()	Output means not near zero
Extreme tilt (> 30 deg)	warning()	AP or ML angle exceeds threshold
Missing dependency	stop() / auto-source	Tries sourcing from working directory

8. Critical Design Decisions

SF re-detection override. When apply_tilt_correction=TRUE, any externally provided step_freq_hz is ignored and SF is re-detected on the tilt-corrected vertical signal. This prevents a subtle 1-FFT-bin shift (~0.003 Hz) that may occur when SF is detected on raw (uncorrected) data. The shift cascades through truncation length, resampling ratio, and produces cumulative phase drift. When apply_tilt_correction=FALSE (pre-corrected data), the provided SF is used directly.

Norm computed before tilt correction. The vector norm is computed in Step 0 from raw signals, before Step 1 applies tilt correction. Mathematically, the norm is invariant to rotation (||Rv|| = ||v||), so the order does not affect the result. The specific order is preserved solely for reproducibility with MATLAB outputs.

Automatic problematic-ratio handling. The MATLAB code hardcoded 8 specific outdoor-condition subject IDs that needed a 2/3 pre-downsampling step before the final resample. The R implementation handles this automatically in resample_signal() by detecting when |ratio - 1| < threshold and applying two-stage resampling. This is more robust (no hardcoded IDs) and generalizes to any dataset.

Target samples from config vs steps. If target_samples is NULL (default), it is computed as target_steps * samples_per_step (250*75=18750). The explicit target_samples parameter allows the batch script to pass different values for non-standard conditions.

S3 class for output. The return value is an S3 gait_preprocessed object with custom print() and summary() methods, providing formatted console output without requiring explicit format calls.

10. Dependencies

Script	Function	Purpose
`tilt_correction.R`	`tilt_correction()`	Moe-Nilssen 1998 tilt correction
`step_frequency_detection.R`	`step_frequency_fft()`	FFT-based cadence detection
`resample_signal.R`	`resample_signal()`	Bandlimited sinc interpolation
`resample_signal.R`	`resample_log()`	Optional debug logging

All dependencies are auto-sourced from the working directory if not already loaded. In the batch pipeline, they are pre-loaded by run_gait_analysis_batch.R.

11. References

Moe-Nilssen, R. (1998). A new method for evaluating motor control in gait under real-life environmental conditions. Part 1: The instrument. Clinical Biomechanics, 13(4-5), 320-327.

Smith, J. O., & Gossett, P. (1984). A flexible sampling-rate conversion method. In Proceedings of ICASSP '84. IEEE International Conference on Acoustics, Speech, and Signal Processing.

Piergiovanni, S., & Terrier, P. (2024). Effects of metronome walking on long-term attractor divergence and correlation structure of gait. Scientific Reports, 14, 15784.

Piergiovanni, S., & Terrier, P. (2024). Validity of linear and nonlinear measures of gait variability to characterize aging gait with a single lower back accelerometer. Sensors, 24(23), 7427.

6. resample_signal.R

MATLAB-Compatible Polyphase Signal Resampling

1. Purpose

`resample_signal.R` resamples truncated gait acceleration signals to a standardized length using polyphase filtering, replicating MATLAB's `resample(x, p, q, N, beta)` algorithm exactly. The function is called by `truncate_resample.R` for each axis (x, y, z, norm) after signal truncation. In the standard ACIER pipeline, this converts truncated signals to 18,750 samples (250 steps at 75 samples/step).

The implementation bypasses the `gsignal::resample()` R wrapper and calls `gsignal::upfirdn()` (compiled C) directly, with a custom sinc + Kaiser filter that replaces the computationally expensive firls matrix solve. Filter caching across axes provides further speedup in batch processing.

2. Scientific Background

Nonlinear gait metrics (LDS, ACI) require fixed-length signals for phase space reconstruction. Since different subjects walk at different cadences, the truncated signals have variable lengths (typically 15,000-40,000 samples). Resampling standardizes these to a common length while preserving the signal's spectral content up to the Nyquist frequency of the shorter signal.

2.1 Algorithm: MATLAB uniformResample()

The algorithm operates in 9 steps:

Step	Operation	Description
1	GCD reduction	Reduce p/q to coprime integers to minimize filter size
2	Filter design	sinc + Kaiser window (equivalent to MATLAB firls with zero-width transition)
3-6	Delay computation	nZeroBegin, Lhalf adjustment, output delay for phase alignment
7	Zero-padding	Pad filter to ensure correct output length
8	upfirdn()	Polyphase filtering via compiled C code (gsignal)
9	Extraction	y = y_full[delay + (1:Ly)] -- trim to exact output length

2.2 Method Selection

Method	Behavior	Notes
`direct`	(default)	Single-step polyphase resampling. Works for all ACIER signals. The batch pipeline forces this mode.
`auto`	Two-stage only when needed	Falls back to two-stage when filter_phases exceeds 50,000 (near integer overflow). The .would_overflow() guard provides a hard safety check.
`two_stage`	Always two-stage	Stage 1: shrink to 2/3. Stage 2: resample to target. Only used when it improves the ratio (P4 smart check).

3. Function Reference

3.1 resample_signal()

Main entry point. Analyzes the ratio, selects method, and executes resampling.

Parameter	Type	Default	Description
signal	numeric	(required)	Input signal vector
target_length	integer	(required)	Desired output length
method	character	"auto"	"auto", "direct", or "two_stage"
complexity_threshold	integer	50000	Max filter phases for auto mode
n	integer	20	Filter half-order (ACIER default)
beta	numeric	5	Kaiser window shape parameter
debug	logical	TRUE	Print console diagnostics
use_fallback	logical	TRUE	Fall back to signal/spline if needed
log_file	character	NULL	Path to debug log file
log_label	character	""	Label for this call (e.g., "S01_x")

Output: numeric vector of length target_length with metadata attributes:

Attribute	Description
method_used	"direct", "two_stage", or "none"
original_length	Length of the input signal
filter_phases	max(p_reduced, q_reduced) after GCD reduction
filter_length	Number of FIR filter taps
was_problematic	TRUE if filter_phases > threshold (diagnostic only)
backend	"gsignal", "signal", "spline", or "none"
elapsed_sec	Wall-clock time for the resampling operation

3.2 check_resampling_ratio()

Diagnostic utility that analyzes the resampling ratio without performing resampling. Returns GCD, reduced p/q, filter phase count, estimated upfirdn buffer size, and the `is_problematic` flag. The default threshold of 50,000 filter phases is set to trigger only near the actual integer overflow limit for typical gait signals.

3.3 Logging Functions

Function	Purpose
resample_log_header(log_file)	Write header with timestamp and R session info. Call ONCE before batch processing. Overwrites the file.
resample_log_sep(log_file, label)	Write a visible separator block. Called by truncate_resample.R per file.
resample_log(log_file, fmt, ...)	Write a single formatted line (sprintf syntax). For custom annotations.

3.4 Internal Functions

Function	Description
.resample_matlab(x, p, q, n, beta)	Direct MATLAB uniformResample() port. Calls upfirdn via gsignal.
.design_matlab_resample_filter(p, q, n, beta)	sinc + Kaiser filter design with caching. O(N) complexity.

`.resample_bandlimited(x, p, q)`	Fallback via signal::resample().
`.resample_spline(x, target_length)`	Last-resort spline interpolation.
`gcd(a, b)`	Euclidean GCD algorithm.
`clear_resample_cache()`	Clears the filter cache environment.

4. Debug Logging Guide

4.1 Two-Tier Architecture

Console output (debug=TRUE): Compact one-line-per-topic summary printed for the first axis (x) only. Controlled by the `debug` parameter in resample_signal(). In the batch pipeline, truncate_resample.R sets debug=TRUE for x-axis and debug=FALSE for y, z, norm.

File log (log_file): Detailed trace appended for ALL axes. Contains signal statistics, ratio analysis, filter design details, delay compensation values, upfirdn buffer sizes, extraction indices, and output statistics. Initialized by resample_log_header() and accumulates across the entire batch run.

4.2 Log File Structure

Each file block in the log contains:

Section	Content
FILE header	File index, name (from truncate_resample.R)
SF / Truncation	Step frequency, truncation length (from truncate_resample.R)
Per-axis block (x4)	Parameters, input stats, ratio analysis, filter info, delay, upfirdn, output stats
RESULT line	method, backend, output length, elapsed time

4.3 Interpreting Key Log Lines

Log Line	Meaning
`GCD=10, reduced p/q=1875/2909`	The coprime ratio after GCD reduction. Large values mean more filter taps.
`filter_phases=2909`	max(p_red, q_red). Determines filter order = 2 * n * phases.
`upfirdn_buffer=54658236 (417.0 MB)`	Memory needed for the polyphase operation. Safe if < 2 GB.
`Filter: from cache [...]`	Filter reused from previous axis (same file, same ratio).
`delay=21, Ly=18750`	Phase alignment delay and exact output length before extraction.
`problematic=TRUE`	Filter phases exceed threshold. In v3.5 (threshold=50000), this rarely triggers.

4.4 Enabling Debug Logging in Batch Mode

In run_gait_analysis_batch.R:

```
resample_log_path <- file.path(config$output_folder, "resample_log.txt")
resample_log_header(resample_log_path)   # once at batch start

prep <- truncate_resample(
    ...,
    resampling_method = "direct",
    log_file = resample_log_path          # enables per-axis logging
)
```

Set log_file = NULL to disable file logging (reduces I/O overhead).

5. Pipeline Integration

The batch script (run_gait_analysis_batch.R) forces method="direct" on line 309. This is correct: MATLAB uses direct resampling for all ACIER signals, and the full 234-file batch completes with 100% success rate in direct mode. The "auto" and "two_stage" methods exist as safety nets for datasets with unusual ratios but are not needed for ACIER.

Filter caching means the first axis per file pays the filter design cost (~0.01 sec with sinc+Kaiser); subsequent axes (y, z, norm) reuse it. Total per-file resampling time is typically 0.1-0.4 seconds across all 4 axes.

6. Error Handling

Condition	Response	Notes
Non-numeric signal	`stop()`	Type validation on input
signal length < 2	`stop()`	Minimum length check
Non-finite values	`warning()`	NA/NaN/Inf propagate but are flagged
equal lengths	`return input`	No resampling needed (method_used = "none")
|sin(theta)| clamping	`clamp`	For resampling, not applicable (tilt_correction handles this)
Integer overflow (auto)	`two-stage or spline`	.would_overflow() triggers fallback
Integer overflow (direct)	`spline fallback`	Last resort if direct would overflow
upfirdn failure	`tryCatch -> spline`	Catches any gsignal errors
Output length mismatch	`trim/extend`	ceil() rounding handled post-resampling

7. Methodological Notes

sinc vs firls. MATLAB's firls() with a zero-width transition band [0 2fc 2fc 1] converges to the ideal lowpass sinc function. Computing sinc directly is O(N) vs O(N^3) for the matrix solve in firls. For the ACIER 75/128 ratio (order=5120), this reduces filter design from 5-10 minutes to under 0.01 seconds. After Kaiser windowing, results are near-identical.

Threshold rationale (v3.5). The default complexity_threshold was raised from 100 to 50,000 based on full-batch evidence: all 936 ACIER signals completed successfully in direct mode with filter phases up to 16,543 (buffer ~2.4 GB, 14.6% of R's integer limit). The `.would_overflow()` function provides the real hard safety check against R's 2^31-1 integer index limit. The threshold now only triggers for genuinely extreme ratios near the overflow boundary.

Two-stage strategy (v3.6). When the resampling ratio is close to 1.0, the GCD of (*target_length, original_length*) can be very small (even 1), producing a polyphase filter with millions of taps and an upfirdn buffer that exceeds R's 2^31 integer index limit. The two-stage strategy avoids this by first shrinking the signal to 2/3 of its length, then resampling the shortened signal to the target. This matches the MATLAB ACIER pipeline, which calls resample(x, 2, 3, 20) for subjects with problematic ratios (outdoor condition only).

A critical implementation detail is that Stage 1 must pass the *ratio* p=2, q=3 directly to the polyphase filter — not the computed lengths (*intermediate_length, original_length*). With the ratio, GCD(2,3)=1 and the reduced filter has max(2,3)=3 phases, order 2×20×3=120, producing a trivial 121-tap filter regardless of signal length. Prior to v3.6, the code passed computed lengths instead, which — when the original length was not divisible by 3 — left the GCD at 1 and produced p_reduced equal to the full intermediate length (e.g., 50,196 for a 75,295-sample outdoor signal). The resulting multi-million-tap filter and ~3.8 billion-element buffer overflowed R's integer limit, causing the "invalid 'times' argument" error in rep(). Indoor data (250 steps, ~37,000 samples) was unaffected because even worst-case buffers

remained below 2^31; outdoor data (500 steps, ~75,000 samples) systematically triggered the overflow.

The P4 smart check verifies that Stage 2 (shortened signal → target) has fewer filter phases than direct resampling before committing to two-stage. If not, it falls back to direct. This prevents two-stage from being worse than direct, which can happen when the intermediate length has a poor GCD with the target. The intermediate length is computed as ceiling(original_length × 2/3), matching MATLAB's ceil(Lx × p/q) output convention for resample(x, 2, 3).

Filter caching. The filter is cached by (p_reduced, q_reduced, n, beta) in a private environment. Since all axes of the same file share the same truncation length and target, they share the same ratio and filter. This saves ~75% of filter design time in batch mode. Call clear_resample_cache() between subjects if memory is a concern.

8. References

MATLAB resample(): https://www.mathworks.com/help/signal/ref/resample.html

Crochiere, R.E. & Rabiner, L.R. (1983). Multirate Digital Signal Processing. Prentice-Hall.

Piergiovanni, S. & Terrier, P. (2024). Validity of linear and nonlinear measures of gait variability to characterize aging gait with a single lower back accelerometer. Sensors, 24(23), 7427.

7. compute_rms.R

Movement Intensity and Lateral Stability Index

1. Purpose

The **compute_rms()** function computes two clinically relevant gait metrics from trunk-worn accelerometer data: **RMS movement intensity** (a proxy for walking speed) and the **RMS ratio** (a speed-independent index of lateral trunk control). These metrics are derived from truncated, preprocessed acceleration signals and are among the simplest yet most discriminating gait biomarkers for fall risk assessment in older adults.

2. Scientific Background

Root Mean Square (RMS) of trunk acceleration quantifies the average magnitude of oscillatory movement during walking. Higher RMS values indicate more vigorous gait, typically associated with faster walking speed (Moe-Nilssen, 1998). Because RMS is strongly speed-dependent, direct comparison between individuals walking at different speeds requires normalization.

The **RMS ratio** addresses this limitation by expressing mediolateral (ML) acceleration as a proportion of total movement intensity: RMS_ratio = RMS_ML / RMS_norm. This ratio is largely independent of walking speed (Sekine et al., 2013) and captures lateral trunk sway, making it a sensitive marker of balance impairment and gait abnormality. In the ACIER study (Piergiovanni & Terrier, 2024), RMS movement intensity was the strongest discriminator between older and younger adults (Hedges' g = -0.58).

3. Function Signature and Parameters

```
result <- compute_rms(az, norm_signal)
```

Parameter	Type	Description
az	Numeric vector	Mediolateral acceleration in g, after tilt correction
norm_signal	Numeric vector	Vector norm with gravity removed, computed from RAW (pre-tilt-correction) signals: norm = sqrt(x^2 + y^2 + z^2) - 1

To replicate ACIER methodology, the **norm_signal** must be computed from raw signals before tilt correction, while **az** must be tilt-corrected.

4. Output

The function returns a named list with two elements:

Field	Units	Description	Typical Range
rms_norm	g	RMS of vector norm (movement intensity)	0.15 - 0.60 g
rms_ratio	dimensionless	RMS_ML / RMS_norm (lateral stability index)	0.40 - 0.90

Higher **rms_norm** values indicate faster walking. Higher **rms_ratio** values indicate greater lateral sway relative to total movement, which may reflect balance deficits.

5. Usage Example

```
# Typical usage within the RACING pipeline
```

```
# (az is tilt-corrected ML; norm is from raw signals)
result <- compute_rms(preprocessed$truncated$z,
                      preprocessed$truncated$norm)

cat("Movement intensity:", result$rms_norm, "g\n")
cat("Lateral stability:",  result$rms_ratio, "\n")
```

6. Pipeline Integration

Within the RACING batch pipeline (**`run_gait_analysis_batch.R`**), **`compute_rms()`** is called automatically after tilt correction and signal truncation. The function operates on truncated signals at the original sampling rate (typically 256 Hz for the ACIER dataset). No resampling is required for these linear metrics.

The output fields map to the results Excel file as follows: **`rms_norm`** is written to the **RMS_norm** column and **`rms_ratio`** to the **RMS_ratio_ml_norm** column.

7. Error Handling

The function includes the following safeguards:

Condition	Behavior
Non-numeric input	Stops with error message
Unequal vector lengths	Stops with error message
Empty input vectors	Stops with error message
NA values in input	Emits warning; returns NA
Zero norm signal (stationary sensor)	Emits warning; rms_ratio set to NA

8. Methodological Notes

8.1 Norm computation order

The vector norm is computed from raw (non-tilt-corrected) accelerations because the Euclidean norm is rotation-invariant: rotating the coordinate frame does not change the magnitude sqrt(x^2 + y^2 + z^2). Gravity is removed by subtracting 1 g from the norm. This approach captures total dynamic movement intensity regardless of sensor orientation.

8.2 Difference from Sekine's RMSR

Sekine et al. (2013) define RMSR = RMS_direction / RMS_T, where RMS_T = sqrt(RMS_AP^2 + RMS_ML^2 + RMS_V^2), computed from the three axis-specific RMS values without gravity subtraction. The ACIER implementation instead uses the sample-wise gravity-subtracted norm. Both approaches produce a dimensionless lateral sway index, but the absolute values differ. Users should not compare RACING output directly with Sekine's published normative values.

8.3 Interpretation guidelines

RMS movement intensity is a proxy for walking speed and should be interpreted alongside measured walking speed when available. In the ACIER cohort, older adults showed a non-significant trend toward higher RMS ratio (0.66 vs. 0.65), while their movement intensity was substantially lower (0.29 vs. 0.35 g).

9. References

[1] Moe-Nilssen, R. (1998). A new method for evaluating motor control in gait under real-life environmental conditions. Part 2: Gait analysis. Clin Biomech, 13(4-5), 328-335.

[2] Sekine, M., et al. (2013). A gait abnormality measure based on root mean square of trunk acceleration. J NeuroEngineering Rehabil, 10, 118.

[3] Piergiovanni, S., & Terrier, P. (2024). Validity of linear and nonlinear measures of gait variability to characterize aging gait with a single lower back accelerometer. Sensors, 24(23), 7427.

8. acf_gait_regularity.R

ACF-Based Gait Regularity

1. Purpose

This script computes step and stride regularity from trunk accelerometer data using the autocorrelation function (ACF). It provides the complete workflow from raw norm signal to publication-ready regularity metrics, including visualization and diagnostic output. The script contains five functions organized in three layers: two low-level helpers (`peakn`, `xcorr_unbiased`), one main computation function (`compute_gait_regularity`), and two user-facing output functions (`plot_gait_regularity`, `print_gait_regularity`).

2. Scientific Background

The autocorrelation method for trunk accelerometry was introduced by Moe-Nilssen & Helbostad (2004). During walking, the acceleration signal repeats with two characteristic periods: the step period (left-right alternation) and the stride period (full gait cycle). The unbiased ACF of the acceleration vector norm reveals these periodicities as peaks. The first dominant peak corresponds to the step period and the second to the stride period; their heights measure gait regularity.

Following Auvinet et al. (2002), the Fisher z-transform (atanh) is applied to the raw correlation coefficients. This variance-stabilizing transformation maps bounded values [-1, 1] to an unbounded scale where equal increments have equal statistical weight, enabling parametric testing. Typical Fisher-transformed values for normal walking range from 1.1 to 1.4 for step regularity and 1.2 to 1.4 for stride regularity, with lower values indicating more variable, less regular gait (Piergiovanni & Terrier, 2024).

3. Function Reference

3.1 compute_gait_regularity() -- Main Analysis

Primary function. Computes step and stride regularity from a preprocessed acceleration norm signal by computing the unbiased ACF, detecting peaks with the n-order criterion, and optionally applying the Fisher z-transform.

```
compute_gait_regularity(acc_norm, sampling_freq = 256,
    max_lag_seconds = 2.0, peak_order = 60, apply_fisher = TRUE)
```

Parameter	Default	Description
`acc_norm`	(required)	Gravity-subtracted vector norm: sqrt(ax^2+ay^2+az^2)-1. Truncated signal at original sampling rate (typically 250 steps).
`sampling_freq`	256	Sampling frequency in Hz. Must match the recording device.
`max_lag_seconds`	2.0	Maximum ACF lag in seconds. At 256 Hz: 512 samples. Batch pipeline overrides with acf_max_lag (500) from config.
`peak_order`	60	Peak detection window half-width (samples). At 256 Hz, 60 samples = 234 ms, preventing false peaks within one gait event.
`apply_fisher`	TRUE	Apply Fisher z-transform (atanh). Values clamped to +/-0.9999 before transformation to prevent Inf.

Returns a list (13 fields):

Field	Type	Description
`step_regularity`	numeric	1st ACF peak (Fisher z-transformed if apply_fisher=TRUE). Primary output metric.
`stride_regularity`	numeric	2nd ACF peak (Fisher z-transformed). Primary output metric.

step_regularity_raw	numeric	Raw correlation coefficient of 1st peak, range [-1, 1].
stride_regularity_raw	numeric	Raw correlation coefficient of 2nd peak, range [-1, 1].
step_lag_samples	integer	Lag of 1st peak in samples (0-indexed).
stride_lag_samples	integer	Lag of 2nd peak in samples (0-indexed).
step_period_seconds	numeric	Step period = step_lag / sampling_freq.
stride_period_seconds	numeric	Stride period = stride_lag / sampling_freq.
acf_values	vector	Full ACF sequence (lags 0 to max_lag). Used by plot_gait_regularity().
peaks	data.frame	All detected peaks: columns index (1-based position) and value.
parameters	list	Echo of all input parameters plus derived values (max_lag_samples, n_samples).

3.2 plot_gait_regularity() -- ACF Visualization

Creates a publication-quality plot of the autocorrelation function with annotated step and stride peaks. The plot shows the full ACF curve in dark blue, with the step peak marked as a green filled circle and the stride peak as a red filled circle. Each peak is labeled with its raw correlation value. A legend in the top-right corner displays both raw and z-transformed values alongside the estimated period in seconds. A dashed horizontal reference line at zero aids interpretation.

```
plot_gait_regularity(result, show_all_peaks = FALSE,
    title = NULL, time_axis = TRUE)
```

Parameter	Default	Description
result	(required)	Output list from compute_gait_regularity(). Must contain acf_values, peaks, and parameters fields.
show_all_peaks	FALSE	If TRUE, marks all detected peaks as open gray circles in addition to the step/stride highlights. Useful for debugging peak detection behavior.
title	NULL	Custom plot title. If NULL, uses default: "ACF-Based Gait Regularity Analysis".
time_axis	TRUE	If TRUE, x-axis shows lag in seconds; if FALSE, in raw sample counts. Seconds recommended for publication figures.

Plot features:

ACF curve: Dark blue line (lwd=1.5) showing autocorrelation from lag 0 to max_lag.

Step peak: Green filled circle (pch=19, cex=2) at the 1st detected peak, labeled with raw correlation.

Stride peak: Red filled circle at the 2nd detected peak, labeled with raw correlation.

All peaks mode: When show_all_peaks=TRUE, all peakn-detected maxima appear as open gray circles (pch=1). This diagnostic mode helps verify that peak_order is appropriate for the signal.

Legend: Top-right box showing raw correlation, Fisher z-value, and period for both step and stride.

Graphics safety: The function saves par() on entry and restores on exit via on.exit(). Margins are set to mar=c(4,4,3,1) for RStudio compatibility. Returns the result object invisibly to allow chaining.

3.3 print_gait_regularity() -- Console Summary

Prints a formatted text summary of gait regularity results to the R console. Designed for interactive exploration and quick quality checking during batch processing development. The output is organized in four sections: step regularity, stride regularity, peak detection summary, and analysis parameters.

```
print_gait_regularity(result)
```

Console output includes:

Step Regularity block: Raw correlation (4 decimal places), Fisher z-transform value, and period in seconds with sample count.

Stride Regularity block: Same three values for the stride (2nd peak).

Detected Peaks: Total number of peaks found by peakn(), useful for diagnosing whether peak_order is too restrictive (too few peaks) or too permissive (too many).

Parameters Used: Echo of sampling frequency, max lag (seconds and samples), peak order, and signal length. This allows verifying that the correct configuration was applied.

The function returns the result object invisibly, enabling chaining: print_gait_regularity(result) |> plot_gait_regularity().

3.4 peakn() -- N-Order Peak Detection

Low-level helper ported from MATLAB peakn.m (attributed to K. Aminian, EPFL Lausanne). A point is classified as a peak if and only if it is the strict maximum within a symmetric window of +/-n samples. Peaks are returned in order of appearance (by position index), not by amplitude. This temporal ordering is critical for the ACF application, where the 1st and 2nd peaks correspond to step and stride periods respectively.

```
peakn(x, n)
```

Parameter	Description
x	Numeric vector to search for peaks (typically the normalized ACF).
n	Window half-width in samples. Valid detection range: positions n+1 to length(x)-n.

Returns a data.frame with columns: index (1-based position in x) and value (amplitude at that position). Uses which.max() internally; ties are resolved by taking the first occurrence, matching MATLAB behavior.

3.5 xcorr_unbiased() -- Unbiased Autocorrelation

Low-level helper replicating MATLAB xcorr(x, maxlag, 'unbiased'). Computes the autocorrelation for positive lags only (0 to max_lag), using the unbiased estimator r(k) = (1/(n-k)) * sum(x[t]*x[t+k]). The signal is NOT mean-centered before computation, matching MATLAB behavior exactly. When normalize=TRUE (default), all values are divided by r(0) to produce correlation coefficients in [-1, 1].

```
xcorr_unbiased(x, max_lag, normalize = TRUE)
```

Parameter	Description
x	Numeric vector. NOT mean-centered (matching MATLAB xcorr behavior).
max_lag	Maximum lag in samples. Must be < length(x).
normalize	If TRUE (default), divide by r(0) to get correlation coefficients. Set FALSE for raw covariances.

Returns a numeric vector of length max_lag+1 (lags 0, 1, ..., max_lag). The unbiased estimator prevents artificial ACF decay at large lags that would bias peak detection.

4. Usage Example

```
source("acf_gait_regularity.R")
```

```
# After preprocessing with truncate_resample():
result <- compute_gait_regularity(
    acc_norm = prep$truncated$norm,
    sampling_freq = 256, peak_order = 60)

# Console summary (quality check)
print_gait_regularity(result)

# Publication figure
plot_gait_regularity(result, time_axis = TRUE)

# Debugging: show all detected peaks
plot_gait_regularity(result, show_all_peaks = TRUE,
    title = "Subject 12 - NC condition")
```

5. Pipeline Integration

In `run_gait_analysis_batch.R`, gait regularity is computed on the truncated norm signal at the original sampling rate (without resampling). The config file controls two parameters: acf_max_lag (row 39, default 500 samples) and peak_min_distance (row 40, default 60 samples). Results are written to output Excel columns G (step_reg) and H (stride_reg).

Important: The norm signal must be computed from RAW (non-tilt-corrected) acceleration as sqrt(ax^2+ay^2+az^2)-1, making it orientation-independent. This matches the ACIER methodology where tilt correction affects individual axes but the vector norm is computed beforehand.

6. Error Handling

Condition	Type	Behavior
Non-numeric input	Error (stop)	Halts with descriptive message in all three public functions.
Signal too short	Error (stop)	Requires > max_lag + peak_order + 1 samples.
NA values in signal	Warning	NAs removed with warning; shortened signal used for ACF.
Fewer than 2 peaks	Warning	Returns NA for all regularity metrics. ACF values and peak list still returned for diagnostics.
Correlation near +/-1	Handled	Clamped to +/-0.9999 before atanh() to prevent Inf output.
Invalid result to plot	Error (stop)	plot_gait_regularity() checks for required fields (acf values, parameters).

7. Debugging and Diagnostics

Several features support troubleshooting and quality control:

print_gait_regularity(): Quick console check of all metric values and parameters. Run after each compute call during development to verify results are within expected ranges.

show_all_peaks mode: In plot_gait_regularity(), setting show_all_peaks=TRUE displays all peaks detected by peakn() as open gray circles. If too many peaks appear (>10), peak_order may be too small; if fewer than 2 appear, peak_order may be too large for the walking speed.

ACF values in output: The full acf_values vector is returned in the result list, enabling custom plotting, comparison against MATLAB reference outputs, or export to CSV for external analysis.

Parameter echo: The parameters sub-list in the result echoes all input parameters including derived values (max_lag_samples, n_samples), enabling post-hoc verification of analysis settings.

NA propagation: When peaks cannot be detected, all regularity fields are set to NA_real_ (not NaN or 0), ensuring clean identification in batch output spreadsheets and downstream statistical analysis.

8. Methodological Notes

Unbiased ACF estimator: Divides by (n-k) at each lag k, compensating for decreasing overlap. This prevents artificial decay that would reduce peak heights at stride-period lags. Matches MATLAB xcorr 'unbiased' exactly.

No mean-centering: MATLAB xcorr does not subtract the signal mean before computing cross-products. The R implementation replicates this behavior. Normalization by r(0) implicitly handles centering for correlation purposes.

Peak ordering: Peaks are returned by temporal position (lag), not amplitude. This is essential because the 1st-by-lag peak is the step peak and the 2nd is the stride peak, regardless of which has higher correlation.

Fisher z-transform clamping: Values are clamped to +/-0.9999 before atanh() to handle edge cases where numerical precision could push correlations to exactly +/-1.0, which would produce +/-Inf.

9. References

[1] Moe-Nilssen R, Helbostad JL. Estimation of gait cycle characteristics by trunk accelerometry. J Biomech. 2004;37(1):121-6.

[2] Auvinet B, Berrut G, Touzard C, et al. Reference data for normal subjects obtained with an accelerometric device. Gait Posture. 2002;16(2):124-34.

[3] Fisher RA. On the probable error of a coefficient of correlation deduced from a small sample. Metron. 1921;1:3-32.

[4] Piergiovanni S, Terrier P. Validity of linear and nonlinear measures of gait variability to characterize aging gait with a single lower back accelerometer. Sensors. 2024;24(23):7427.

9. ami.R

Average Mutual Information for Optimal Embedding Delay

1. Purpose

`ami.R` determines the optimal time delay (tau) for phase space reconstruction, a critical prerequisite for computing Local Dynamic Stability (LDS) and the Attractor Complexity Index (ACI) via Rosenstein's divergence algorithm. The script computes Average Mutual Information (AMI) between a signal and its time-delayed copies over a range of lags. The first minimum of the AMI curve identifies the delay at which the lagged signal provides maximum independent information about the original -- the theoretically optimal embedding delay (Fraser & Swinney, 1986). The script is a faithful port of four MATLAB functions by Durga Lal Shrestha (2005), deliberately preserving an internal binning inconsistency present in the MATLAB original to ensure numerical compatibility.

2. Scientific Background

2.1 Phase Space Reconstruction

Takens' embedding theorem (1981) states that a dynamical system's attractor can be reconstructed from a single scalar time series by creating time-delayed copies. Given a signal x(t), the m-dimensional embedding vector is Y(t) = [x(t), x(t+tau), x(t+2*tau), ..., x(t+(m-1)*tau)], where m is the embedding dimension and tau is the embedding delay. The quality of the reconstruction depends critically on tau: too small makes adjacent coordinates nearly identical (redundant), too large makes them effectively independent (loss of dynamic coupling).

2.2 Why AMI Over Autocorrelation?

The autocorrelation function (ACF) only captures linear dependencies. AMI captures both linear and nonlinear relationships by measuring the information shared between x(t) and x(t+tau) in an information-theoretic sense. The first minimum of the AMI curve indicates where the lagged signal provides the most new information while retaining enough coupling to preserve the attractor topology. For gait acceleration signals, typical optimal delays are 5-20 samples.

2.3 Mutual Information Definition

For discrete probability distributions, mutual information is defined as: I(X;Y) = Sum p(x,y) * log2(p(x,y) / (p(x) * p(y))), where p(x), p(y) are marginal probability distributions and p(x,y) is the joint probability distribution. The result is measured in bits. At lag 0, I equals the signal's entropy (maximum). As lag increases, I typically decreases, reaching a first minimum at the optimal delay.

2.4 Deliberate MATLAB Binning Inconsistency

The original MATLAB code by Shrestha (2005) uses two different binning strategies internally: center-based bins for 1D marginals (via MATLAB hist(), bin_width = range/(M-1)) and edge-based bins for 2D joint probabilities (via histc(), bin_width = range/M). With 64 bins, this creates a ~1.6% difference in bin width between the marginal and joint distributions. This R port deliberately replicates this inconsistency to ensure AMI values match the MATLAB output exactly.

2.5 The Tau Convention

The AMI curve ami_values has index 1 = lag 0, index 2 = lag 1, and so on. When the minimum is at position k (1-based), the actual lag is k-1. However, MATLAB's convention in the ACIER pipeline uses find(diff(daa)>0)(1), which returns the 1-based position k. This k is passed directly to lyarosenstein() as tau. The R implementation preserves this exact convention: find_first_minimum() returns k, and rosenstein_divergence() receives k as tau. This is consistent throughout the entire RACING pipeline.

3. Function Reference

3.1 ami() -- Main Function

Computes AMI and correlation for a range of lags. Returns the full AMI curve and the optimal embedding delay.

Parameter	Type	Default	Description
`signal`	numeric/matrix	`(required)`	Univariate time series vector, or 2-column matrix for bivariate analysis
`n_bins`	integer	`64`	Number of bins for probability estimation (ACIER default: 64)
`n_lags`	integer	`100`	Number of lags to compute (ACIER default: 100)
`plot`	logical	`FALSE`	Create diagnostic dual-axis plot (AMI + correlation)

Return value (list):

Component	Type	Description
`ami`	numeric(n_lags+1)	AMI values for lags 0 through n_lags (in bits)
`correlation`	numeric(n_lags+1)	Pearson correlation for lags 0 through n_lags
`lags`	integer(n_lags+1)	Lag indices: 0, 1, 2, ..., n_lags
`optimal_lag`	integer	First minimum of AMI curve (1-based index = MATLAB tau convention)

3.2 find_first_minimum() -- Optimal Delay Detection

Finds the first local minimum in the AMI curve. Returns a **1-based position** into the AMI values vector, matching MATLAB's convention. If no local minimum exists, falls back to the point where AMI drops below 1/e of its initial value. Returns NA if no suitable delay is found.

Parameter	Type	Default	Description
`ami_values`	numeric	`(required)`	Vector of AMI values (from ami()$ami)

3.3 estimate_delay() -- Convenience Wrapper

Wrapper that returns just the optimal tau value. If no clear minimum is found, defaults to tau=10 with a warning. Suitable for quick one-line usage when the full AMI curve is not needed.

Parameter	Type	Default	Description
`signal`	numeric	`(required)`	Time series vector
`n_bins`	integer	`64`	Number of bins for probability estimation
`max_lag`	integer	`100`	Maximum lag to search

3.4 Internal Helper Functions

R Function	MATLAB Source	Description
`rhist(y, n_bins)`	`rhist.m`	Center-based relative histogram. Uses bin_width = range/(M-1) via seq(length.out=M), matching

		MATLAB hist(y,M). Used for 1D marginal probability distributions.
`has_zero_bins(y, n_bins)`	`isZeroDistribution()`	Checks if any histogram bin has zero frequency. Used by prob_1d() in the adaptive binning bisection loop.
`prob_1d(y, max_bins)`	`prob.m`	Adaptive-bin 1D probability distribution. Uses bisection search to find the maximum number of bins with no empty bins, matching MATLAB prob() exactly.
`prob_2d(x, y, n_bins_x, n_bins_y)`	`probxy.m + computeEdge()`	Edge-based 2D joint probability distribution. Uses bin_width = range/M with -Inf/Inf boundaries and left-closed intervals, matching MATLAB computeEdge() and histc() semantics.
`mutual_info(x, y, n_bins_x, n_bins_y, px, py)`	`(inline in ami.m)`	Computes mutual information in bits (log2) from pre-computed marginal and joint distributions. Skips zero-probability bins.
`ami_plot(lags, ami_values, corr_values)`	`(no MATLAB equivalent)`	Dual-axis diagnostic plot with correlation (left axis, blue) and AMI (right axis, red). Marks the optimal delay with a point and vertical dashed line.
`test_ami()`	`(no MATLAB equivalent)`	Six unit tests covering basic functionality, AMI trend, correlation identity, short signals, MATLAB-style binning, and bivariate input.

4. ACIER Parameters

Parameter	Value	Source
`n_bins`	64	Main_results_new_v2.m: ami(signal, 64, 100)
`n_lags`	100	Main_results_new_v2.m: ami(signal, 64, 100)
`Search clip`	80 (MATLAB) / all (R)	MATLAB: diff(daa(1:80)); R: searches full range
`Default fallback tau`	Error (MATLAB) / 10 (R)	MATLAB crashes; R provides fallback

5. Binning Strategy Details

5.1 1D Marginals: Center-Based (rhist)

MATLAB's `hist(y, M)` creates M bin centers via `linspace(min, max, M)`, giving bin_width = range / (M-1). The R function `rhist()` replicates this with `seq(min_y, max_y, length.out = n_bins)`. Boundary bins extend to -Inf and +Inf through midpoint-based edges.

5.2 2D Joints: Edge-Based (prob_2d)

MATLAB's `computeEdge()` in probxy.m uses edge-based binning: bin_width = range / M (not M-1). Edges are computed as `min + bw * (0:M)`, then first and last are replaced with -Inf/Inf. The R function `prob_2d()` replicates this exactly. The MATLAB histc semantics (left-closed intervals [a,b), last bin includes right boundary) are reproduced via manual comparison loops.

5.3 Adaptive Bin Reduction (prob_1d)

Both MATLAB prob.m and R prob_1d() use a bisection algorithm to find the maximum bin count that produces no empty bins. Starting from the requested n_bins (typically 64), if any bin has zero probability, the count is halved; if all bins are non-empty, the count is increased by bisection between the last-known zero and non-zero counts. This ensures a valid probability distribution for the MI calculation, where log(0) would be undefined.

6. Usage Examples

6.1 Standard ACIER Pipeline Usage

```
source("ami.R")

# Compute AMI on resampled signal (18750 samples)
result <- ami(resampled_signal, n_bins = 64, n_lags = 100)
tau <- result$optimal_lag    # e.g., 7 (1-based position)

# Pass tau to Rosenstein's algorithm
div <- rosenstein_divergence(resampled_signal,
  m = 5, tau = tau, mean_period = 75, max_iter = 1800)
```

6.2 Quick Delay Estimation

```
tau <- estimate_delay(signal)  # Returns integer; defaults to 10 on failure
```

6.3 Batch Processing (Per Axis)

```
axes <- c("x", "y", "z", "norm")
ami_vals <- integer(4)
names(ami_vals) <- axes

for (ax in axes) {
  ami_result <- ami(prep$resampled[[ax]], n_bins = 64, n_lags = 100)
  ami_vals[ax] <- ami_result$optimal_lag
  if (is.na(ami_vals[ax])) {
    ami_vals[ax] <- 10L  # Fallback
    warning(paste0("AMI_", ax, ": no minimum found, using tau=10"))
  }
}
```

6.4 Diagnostic Plot

```
result <- ami(signal, n_bins = 64, n_lags = 100, plot = TRUE)
```

Produces a dual-axis plot with correlation (blue, left axis) and AMI (red, right axis). The optimal delay is marked with a filled dot and a vertical dashed line.

6.5 Running Unit Tests

```
source("ami.R")
test_ami()  # Runs 6 self-tests
```

7. Pipeline Integration

AMI sits between preprocessing and nonlinear dynamics analysis:

Script	Role	Connection
`truncate_resample.R`	Produces resampled signals (18750 samples)	Input to AMI
`ami.R`	Determines optimal tau for each axis (x, y, z, norm)	This script
`rosenstein_divergence.R`	Builds phase space with tau, computes divergence curve	Uses AMI tau
`fit_divergence_exponents.R`	Extracts LDS and ACI from divergence curve	Uses divergence

Critical: tau is axis-specific. Each of the four axes (x, y, z, norm) has its own AMI-determined tau, which is passed to its own Rosenstein divergence call. The embedding dimension (m=5) and mean_period (75) are shared.

8. Downstream Impact of Tau

AMI drives the accuracy of all downstream nonlinear metrics. An incorrect tau distorts the reconstructed phase space attractor, which propagates to the divergence curve and ultimately to both LDS and ACI. Validation has shown that Y-axis AMI is particularly sensitive: a 1-sample error in tau can shift LDS_Y significantly, while ACI is more robust. The vector norm axis tends to be the most stable across both platforms.

9. Error Handling

Condition	Response	Details
Signal is matrix with > 2 columns	stop()	Only univariate or bivariate input accepted
n_lags < 0	stop()	Must be non-negative
n_lags >= signal length	stop()	Not enough samples for requested lags
n_bins < 2	stop()	Minimum 2 bins required
All values identical (constant signal)	Handled	rhist puts all counts in center bin
NA values in AMI curve	warning()	NAs replaced with Inf; results may be unreliable
No AMI minimum found	NA / fallback tau=10	estimate_delay() returns 10 with warning

10. Differences from MATLAB

Feature	MATLAB	R	Notes
Search range	diff(daa(1:80))	All n_lags (100)	Only matters if minimum is beyond lag 79 (rare)
No-minimum handling	Error (index crash)	1/e fallback then NA	R provides graceful degradation
Output format	Two vectors [amis, corrs]	Named list with metadata	R adds optimal_lag, lags fields
Diagnostic plot	Not in ami.m	Built-in ami_plot()	Dual-axis AMI + correlation visualization
Unit tests	Not available	test_ami() with 6 tests	Validates binning, trends, edge cases
Bivariate support	Two-column matrix input	Same + auto-transpose	R adds wide-to-tall conversion

11. Dependencies

This script is fully standalone with no external R package dependencies. It requires only base R (seq, findInterval, tabulate, cor, par, plot). No other RACING scripts are needed. It is self-contained by design -- all helper functions (rhist, prob_1d, prob_2d, mutual_info) are defined within the same file.

12. References

Fraser, A. M. & Swinney, H. L. (1986). Independent coordinates for strange attractors from mutual information. Physical Review A, 33(2), 1134-1140.

Takens, F. (1981). Detecting strange attractors in turbulence. In Dynamical Systems and Turbulence, Warwick 1980. Lecture Notes in Mathematics, vol 898.

Shrestha, D. L. (2005). Average Mutual Information [MATLAB code]. Version 1.1.0.

Piergiovanni, S., & Terrier, P. (2024). Effects of metronome walking on long-term attractor divergence and correlation structure of gait. Scientific Reports, 14, 15784.

Rosenstein, M. T., Collins, J. J., & De Luca, C. J. (1993). A practical method for calculating largest Lyapunov exponents from small data sets. Physica D, 65, 117-134.

10. rosenstein_divergence.R

Rosenstein (1993) Divergence Curve Computation

1. Purpose

`rosenstein_divergence.R` computes logarithmic divergence curves from time series using Rosenstein's (1993) algorithm. For each pair of nearest neighbors in the reconstructed phase space, it tracks how their Euclidean distance evolves over time, then averages the logarithmic distances at each time step (Equation 13 of the paper). The output divergence curve is passed to `fit_divergence_exponents.R` for LDS (short-term stability) and ACI (long-term complexity) extraction.

In the ACIER pipeline, the function processes resampled 18,750-sample signals with embedding dimension m=5, axis-specific time delay tau from AMI, Theiler window of 75 samples (one step), and computes divergence curves up to 1,800 samples (12 strides).

2. Scientific Background

2.1 Rosenstein's Algorithm

The algorithm estimates the largest Lyapunov exponent by tracking the divergence of nearby trajectories in a reconstructed phase space. A positive exponent indicates chaos: nearby states diverge exponentially. In gait analysis, the short-term divergence rate (LDS) quantifies dynamic balance, while the long-term divergence behavior (ACI) reflects attractor complexity.

Step	Operation	Description
1	Phase space reconstruction	Build M x m delay embedding matrix using Takens' theorem: Y(t) = [x(t), x(t+tau), ..., x(t+(m-1)*tau)]
2	Nearest neighbor search	For each point Y(j), find the closest point Y(j*) in Euclidean distance, excluding temporal neighbors within +/- mean_period (Theiler window)
3	Divergence tracking	For k = 1 to max_iter, compute d(k) = <ln ||Y(j+k) - Y(j*+k)||> averaged over all valid pairs (Eq. 13)
4	Slope fitting	Handled separately by fit_divergence_exponents.R: linear regression on d(k) in stride-normalized ranges

2.2 Theiler Window

The temporal exclusion window (Theiler, 1986) prevents selecting nearest neighbors that are simply time-shifted copies of the reference point. For gait signals with strong periodicity, neighbors within one step period (75 samples in ACIER) are excluded. This ensures the nearest neighbor search finds geometrically close trajectories in phase space, not just the same trajectory shifted by a few samples.

2.3 Divergence Interpretation

Metric	Stride range	Sample range	Interpretation
LDS	0 to 0.5 strides	Samples 1-75	Short-term exponential divergence rate; measures dynamic balance
ACI	5 to 12 strides	Samples 750-1800	Long-term divergence slope; measures attractor complexity

3. Function Reference

3.1 rosenstein_divergence()

Main entry point. Estimates mean period (if not provided), performs phase space reconstruction, nearest neighbor search, and divergence tracking.

Parameter	Type	Default	Description
`x`	numeric	`(required)`	Input time series (resampled signal)
`m`	integer	`5`	Embedding dimension
`tau`	integer	`10`	Time delay in samples (from AMI)
`mean_period`	integer	`NULL`	Theiler window; if NULL, estimated from ACF
`max_iter`	integer	`1800`	Maximum divergence tracking steps
`verbose`	logical	`FALSE`	Print progress messages

Returns a list with three elements:

Element	Description
`divergence`	Numeric vector of average log-divergence d(k) for k = 1 to max_iter
`time_steps`	Integer vector 1:length(divergence)
`params`	List of parameters: m, tau, mean period, max iter, N, M

3.2 embed_time_series()

Implements Takens' embedding theorem. Given a scalar time series `x` of length N, embedding dimension `m`, and time delay `tau`, constructs an M x m matrix (M = N - (m-1)*tau) where each row is a delay vector.

Parameters: x (numeric vector), m (integer, embedding dimension), tau (integer, time delay). Returns: M x m matrix.

3.3 find_nearest_neighbors()

For each of the M points in phase space, finds the nearest neighbor in Euclidean distance while excluding temporal neighbors within +/- `mean_period` samples. Uses RANN::nn2() KD-tree when available (O(M log M)); falls back to brute-force pairwise distance computation (O(M^2)). Parameters: Y (M x m matrix), mean_period (integer), verbose (logical). Returns: integer vector of length M.

3.4 compute_divergence()

Implements Equation 13 from Rosenstein et al. (1993): d(k) = <ln(d_j(k))> where d_j(k) = ||Y(j+k) - Y(j*+k)||. For each time step k, computes Euclidean distances between all evolved trajectory pairs, excludes zero distances, and returns the mean of the natural logarithm. The computation is vectorized (all valid pairs per k step). Parameters: Y (M x m matrix), nearest_pos (integer vector), max_iter (integer). Returns: numeric vector of length max_iter.

3.5 estimate_mean_period()

Estimates the dominant periodicity from the autocorrelation function for use as the Theiler window. Strategy: (1) first zero crossing of ACF, (2) first local minimum of ACF, (3) fallback to N/20. In the ACIER batch pipeline, this function is not called because mean_period is provided explicitly from the configuration (75 samples = one step).

4. Usage Examples

Standalone usage with default parameters:

```
source("rosenstein_divergence.R")
result <- rosenstein_divergence(x, m = 5, tau = 17, mean_period = 75, max_iter = 1800)
plot(result$time_steps, result$divergence, type = "l")
```

In the batch pipeline (run_gait_analysis_batch.R):

```
div_result <- rosenstein_divergence(
    x           = prep$resampled[[ax]],
    m           = config$embedding_dim,     # 5
    tau         = ami_vals[ax],              # axis-specific from AMI
    mean_period = config$mean_period,        # 75
    max_iter    = config$divergence_length   # 1800
)
```

Feeding into LDS/ACI fitting:

```
exponents <- fit_divergence_exponents(
    divergence       = div_result$divergence,
    samples_per_step = 75,
    lds_steps        = c(0, 0.5),      # 0-0.5 strides
    aci_stride_range = c(5, 12)        # 5-12 strides
)
```

5. Pipeline Integration

The script operates on resampled signals (18,750 samples = 250 steps x 75 samples/step) after the full preprocessing chain: CSV import, tilt correction, step frequency detection, truncation, and resampling. The AMI-derived time delay (tau) is computed per axis before this function is called. The divergence curve is then passed to fit_divergence_exponents.R for slope extraction.

Processing order per file: AMI (4 axes) -> Rosenstein divergence (4 axes) -> LDS/ACI fitting (4 axes). The four axes are independent and processed in parallel when multiple CPU cores are available (see Chapter 12, Section 3.1). Each axis call is wrapped in tryCatch to handle edge cases gracefully.

6. Error Handling

Condition	Response	Notes
Signal too short for embedding	stop()	M = N - (m-1)*tau must be > 0
mean_period not provided	ACF estimation	estimate_mean_period() called automatically
RANN not available	Brute-force fallback	O(M^2) loop, ~20-40x slower
All NN within Theiler window (RANN)	Per-point brute-force	Rare; Rare; only when all k nearest candidates fall within the Theiler window.
Zero distance at step k	Excluded from mean	Avoids -Inf in log()
No valid pairs at step k	NA	Cleaner than MATLAB's default 0

7. Performance

For ACIER signals (M ~ 18,700 embedded vectors, max_iter = 1800), the nearest neighbor search dominates execution time. With RANN's KD-tree (O(M log M)), total computation is approximately 2-5 seconds per axis. Without RANN (brute-force O(M^2)), it rises to 25-45 seconds per axis. The divergence loop itself runs in 1-3 seconds regardless, since it is vectorized across all valid pairs at each time step k.

Since the four axes are independent, the batch pipeline processes them in parallel using a PSOCK cluster (default: 4 workers). This reduces the per-file nonlinear computation time from approximately 4x the single-axis cost to approximately 1x, yielding a roughly 3-4 fold speedup for the nonlinear analysis stage.

RANN is strongly recommended and is listed as a required dependency for the RACING CodeOcean capsule. Install via: install.packages("RANN").

8. Methodological Notes

Embedding dimension (m=5). Determined by the Global False Nearest Neighbors (GFNN) method in the original ACIER study. Applied uniformly to all signals. A 5-dimensional phase space is standard for lower-back acceleration gait signals.

Time delay (tau). Computed per axis by AMI (Average Mutual Information). Typical values for ACIER resampled signals are 15-18 samples. The delay critically affects LDS but has minimal impact on ACI, because short-term trajectory geometry is sensitive to tau while the global attractor structure is more robust.

Theiler window (mean_period=75). Set to one step duration (75 samples at the resampled rate). This is a conservative choice that effectively eliminates autocorrelation artifacts. The MATLAB implementation uses the same value.

NA vs 0 for empty iterations. When no valid pairs exist at iteration k (very rare, only near the end of the curve), the R implementation returns NA rather than MATLAB's default 0. This prevents 0 from being misinterpreted as a valid log-divergence value in downstream fitting.

9. References

Rosenstein, M. T., Collins, J. J., & De Luca, C. J. (1993). A practical method for calculating largest Lyapunov exponents from small data sets. Physica D, 65(1-2), 117-134.

Takens, F. (1981). Detecting strange attractors in turbulence. In Dynamical Systems and Turbulence, Lecture Notes in Mathematics 898, 366-381. Springer.

Theiler, J. (1986). Spurious dimension from correlation algorithms applied to limited time-series data. Physical Review A, 34(3), 2427-2432.

Piergiovanni, S. & Terrier, P. (2024). Validity of linear and nonlinear measures of gait variability to characterize aging gait with a single lower back accelerometer. Sensors, 24(23), 7427.

11. fit_divergence_exponents.R

LDS and ACI Extraction from Rosenstein Divergence Curve

1. Purpose

`fit_divergence_exponents.R` extracts two gait variability metrics from Rosenstein divergence curves by fitting linear regressions over stride-normalized time ranges. The two metrics are the Local Dynamic Stability (LDS), a short-term divergence exponent measuring dynamic balance, and the Attractor Complexity Index (ACI), a long-term divergence exponent reflecting gait complexity and automaticity. This script receives the raw divergence curve from `rosenstein_divergence.R` and outputs scalar slope values in lambda/stride units.

2. Scientific Background

2.1 Time Normalization

The divergence curve x-axis is normalized by STRIDE duration, not step duration. In the ACIER pipeline, 1 step = 75 samples, 1 stride = 2 steps = 150 samples. For sample index i (1-based), the normalized time is t = (i-1) / 150 strides.

2.2 LDS -- Local Dynamic Stability

LDS measures the short-term exponential divergence rate of nearby trajectories in reconstructed phase space. It quantifies sensitivity to small perturbations during a single gait step (0 to 0.5 strides). Higher LDS values indicate less stable gait and greater fall risk. LDS has been shown to be a consistent marker of dynamic balance across multiple studies (Dingwell & Cusumano, 2000; Terrier & Reynard, 2018). In the ACIER study, LDS was computed for the mediolateral axis only (LDS-ML), as it reflects frontal plane control most directly.

2.3 ACI -- Attractor Complexity Index

ACI measures the long-term divergence slope over 5-12 strides. Originally interpreted as a stability measure, it has been reinterpreted since 2016 as a complexity index. ACI correlates strongly with DFA scaling exponents (r ~ 0.7-0.8), reflecting stride-to-stride fluctuation patterns and gait automaticity. Lower ACI values during metronome walking indicate reduced automaticity. ACI and LDS are conceptually distinct: LDS assesses local instability, while ACI characterizes global attractor structure.

2.4 Fitting Ranges

Metric	Stride range	Sample indices	Time (strides)	Interpretation
LDS	0 to 0.5 strides	1 to 75	0 to 0.493	Short-term: first step
ACI	5 to 12 strides	750 to 1800	4.993 to 11.993	Long-term: stride fluctuations

3. Function Reference

3.1 fit_short_term_divergence()

Computes LDS by fitting a linear regression to the initial portion of the divergence curve.

Parameter	Type	Default	Description
`divergence`	numeric	`(required)`	Divergence curve vector from rosenstein_divergence()
`samples_per_step`	integer	`75`	Samples per gait step (ACIER: 75)

num_steps	integer	1	Number of steps for fitting (1 = 0.5 strides)
return_fit	logical	FALSE	If TRUE, return full fit details (slope, intercept, R^2, lm object)

Returns: If return_fit=FALSE (default), a single numeric value (LDS slope in 1/stride). If return_fit=TRUE, a list with slope, intercept, r_squared, fit (lm object), range_samples, range_strides, and n_points.

3.2 fit_long_term_divergence()

Computes ACI by fitting a linear regression to the later portion of the divergence curve.

Parameter	Type	Default	Description
divergence	numeric	(required)	Divergence curve vector from rosenstein_divergence()
samples_per_step	integer	75	Samples per gait step
stride_range	numeric(2)	c(5, 12)	Fitting range in strides [start, end]
return_fit	logical	FALSE	If TRUE, return full fit details

Returns: Same structure as fit_short_term_divergence(). The ACI slope is in 1/stride units.

3.3 fit_divergence_exponents()

Convenience wrapper that calls both fit_short_term_divergence() and fit_long_term_divergence() from a single divergence curve. This is the function called by the batch pipeline.

Parameter	Type	Default	Description
divergence	numeric	(required)	Divergence curve vector
samples_per_step	integer	75	Samples per gait step
lds_steps	integer	1	Number of steps for LDS (maps to num_steps)
aci_stride_range	numeric(2)	c(5, 12)	ACI fitting range in strides
return_fits	logical	FALSE	If TRUE, include lm objects in output

Returns a list with: LDS (numeric), ACI (numeric), params (list of settings used), and optionally fits (list of full fit objects).

3.4 plot_divergence_fits()

Visualizes the divergence curve with LDS and ACI fitting regions highlighted. LDS region is shown in blue, ACI in red, with dashed regression lines and dotted vertical boundaries. Uses RStudio-compatible margins (par save/restore with on.exit). Returns the fitted exponents invisibly.

4. Usage Examples

Standard ACIER analysis:

```
source("rosenstein_divergence.R")
source("fit_divergence_exponents.R")

# Compute divergence curve
div <- rosenstein_divergence(signal, m=5, tau=17, mean_period=75, max_iter=1800)

# Extract both exponents
exponents <- fit_divergence_exponents(div$divergence)
cat("LDS:", exponents$LDS, "per stride\n")
cat("ACI:", exponents$ACI, "per stride\n")
```

Detailed analysis with fit quality:

```
detailed <- fit_divergence_exponents(div$divergence, return_fits = TRUE)
cat("LDS R^2:", detailed$fits$LDS$r_squared, "\n")
cat("ACI R^2:", detailed$fits$ACI$r_squared, "\n")
```

Visualization:

```
plot_divergence_fits(div$divergence)
```

5. Pipeline Integration

In the batch pipeline (`run_gait_analysis_batch.R`), this script is called once per axis (x, y, z, norm) after `rosenstein_divergence()` computes the divergence curve. The full nonlinear chain (AMI + Rosenstein + fitting) for each axis runs as a single worker function, with all four axes dispatched in parallel. Each call is wrapped in tryCatch, storing NA for any axis that fails. All parameters flow from the Excel configuration through config$samples_per_step, lds_steps, and aci_stride_range.

Processing order per file: AMI (4 axes) -> Rosenstein divergence (4 axes) -> fit_divergence_exponents (4 axes) -> Excel output.

6. MATLAB Correspondence

This script replicates the fitting logic from Main_results_new_v2.m of the ACIER study. The table below maps MATLAB variables to R parameters.

MATLAB variable	Value	R equivalent	Description
`numsamp`	75	`samples_per_step`	Samples per gait step
`numsamp*2`	150	`samples_per_stride`	Samples per stride (computed)
`lds_length`	1	`lds_steps / num_steps`	Steps for LDS fitting
`range_low`	5	`aci_stride_range[1]`	ACI lower bound (strides)
`range_high`	12	`aci_stride_range[2]`	ACI upper bound (strides)
`t`	0:1/150:...	`time_strides`	Stride-normalized time axis
`polyfit(...,1)`	OLS	`lm(div ~ time)`	Linear regression
`pol(1)`	slope	`coef(fit)[2]`	Slope extraction

7. Error Handling

Condition	Response	Notes
Non-numeric divergence	stop()	Input validation
samples_per_step <= 0	stop()	Input validation
Divergence too short for LDS	stop()	Need >= 75 samples for 1 step
Divergence too short for ACI	stop()	Need >= 1800 samples for 5-12 strides
stride_range[1] >= stride_range[2]	stop()	Invalid range order
< 2 valid data points	warning() + NA	Graceful degradation for sparse data

8. Methodological Notes

LDS range (0-0.5 strides). The 0 to 0.5 stride range was selected based on Terrier & Reynard (2018), who showed this range provides the best assessment of dynamic balance. Longer ranges (0-1 stride) mix short-term instability with stride-cycle dynamics.

ACI range (5-12 strides). The 5-12 stride range was determined through systematic sensitivity analysis in the ACIER study (Scientific Reports 2024, Supplementary Table). This range captures the linear portion of the long-term divergence plateau while avoiding the nonlinear transition zone (1-4 strides) and end-of-curve artifacts.

Units: lambda/stride. Both LDS and ACI are expressed in natural log divergence per stride. This differs from some literature that reports LDS in per-second units (lambda/s = slope * fs). The stride normalization makes values directly comparable across studies regardless of sampling frequency.

Typical value ranges. For healthy older adults walking normally (ACIER data): LDS-ML ~ 0.8-1.5 per stride, ACI-Norm ~ 0.02-0.04 per stride, ACI-AP ~ 0.01-0.04 per stride. ACI values can be negative (attractor contraction), which is valid.

R^2 quality thresholds. For well-behaved gait signals, LDS R^2 should exceed 0.95 (the initial divergence is highly linear). ACI R^2 is typically lower (0.85-0.98) because the long-term divergence plateau has more variability.

9. References

Rosenstein, M. T., Collins, J. J., & De Luca, C. J. (1993). A practical method for calculating largest Lyapunov exponents from small data sets. Physica D, 65(1-2), 117-134.

Dingwell, J. B., & Cusumano, J. P. (2000). Nonlinear time series analysis of normal and pathological human walking. Chaos, 10(4), 848-863.

Terrier, P., & Reynard, F. (2018). Maximum Lyapunov exponent revisited: Long-term attractor divergence of gait dynamics is highly sensitive to the noise structure of stride intervals. Gait & Posture, 66, 236-241.

Piergiovanni, S., & Terrier, P. (2024). Effects of metronome walking on long-term attractor divergence and correlation structure of gait. Scientific Reports, 14, 15784.

Piergiovanni, S., & Terrier, P. (2024). Validity of linear and nonlinear measures of gait variability to characterize aging gait with a single lower back accelerometer. Sensors, 24(23), 7427.

12. Batch Pipeline Scripts

read_config.R | import_csv_lowback.R | write_results.R | run_gait_analysis_batch.R

1. Purpose

These four scripts form Layer 3 of the RACING pipeline: the automated batch processing system. Together they handle configuration management, data import, orchestration of all analysis functions, and formatted output generation. They enable researchers to process entire datasets of accelerometer CSV files with a single command, producing a structured Excel workbook of gait quality metrics.

`read_config.R` reads and validates all analysis parameters from a RACING configuration Excel file. `import_csv_lowback.R` imports individual CSV accelerometer files using the configuration settings. `run_gait_analysis_batch.R` orchestrates the full pipeline for every CSV file in the dataset. `write_results.R` formats and saves results to a multi-sheet Excel workbook with comprehensive safety features.

2. Scientific Background

2.1 Pipeline Architecture

The RACING batch pipeline replicates the processing order of the MATLAB ACIER study pipeline. For each accelerometer file, the following steps are executed in order: (1) import raw CSV data, (2) compute vector norm from raw signals (before any correction), (3) apply Moe-Nilssen tilt correction, (4) detect step frequency via FFT on the corrected vertical acceleration, (5) round step frequency to 0.1 Hz, (6) truncate the signal to a target number of steps, (7) resample to a standardized length, (8) compute linear metrics on the truncated signal, and (9) compute nonlinear metrics on the resampled signal.

This order is critical: the norm must be computed before tilt correction (for RMS and regularity calculations), step frequency must be detected after tilt correction (to ensure the FFT operates on properly aligned vertical acceleration), and linear vs nonlinear metrics must use the correct signal version (truncated at original rate vs resampled at standardized rate).

2.2 Dual Signal Outputs

Signal Version	Length	Sampling Rate	Used For
Truncated	floor(fs/SF * N_steps)	Original (e.g. 256 Hz)	RMS, SF, ACF regularity
Resampled	N_steps x 75 samples/step	Standardized	AMI, LDS, ACI

Linear metrics use the original sampling rate. Nonlinear metrics operate on reconstructed phase space attractors and require standardized time scales for valid inter-subject comparison.

2.3 Output Metrics

The pipeline produces 21 columns per file:

Metric	Type	Description
file_id	Integer	Sequential file identifier
subject_id	String	Input CSV filename (without extension)

n_steps	Numeric	Actual number of steps in truncated signal
SF_hz	Numeric	Step frequency from FFT analysis (Hz)
RMS_norm	Numeric	RMS of acceleration norm (movement intensity, g)
RMS_ratio_ml_norm	Numeric	Ratio of ML RMS to norm RMS (lateral stability)
step_reg	Numeric	Step regularity from ACF (Fisher z-transformed)
stride_reg	Numeric	Stride regularity from ACF (Fisher z-transformed)
AMI_x/y/z/norm	Integer	Optimal time delay for phase space reconstruction (samples)
LDS_x/y/z/norm	Numeric	Local dynamic stability (short-term divergence exponent, lambda/stride)
ACI_x/y/z/norm	Numeric	Attractor complexity index (long-term divergence exponent, lambda/stride)
warnings	String	Quality flags and error messages

3. Processing Pipeline

The batch script processes each CSV file through the following steps. Steps A-B produce the dual signal outputs; steps D-G compute metrics on the appropriate signal version.

Step	Operation	Script Used	Signal Version
A	CSV Import	import_csv_lowback.R	Raw
B	Tilt + SF + Truncate + Resample	truncate_resample.R	Both outputs
D	RMS (norm and ratio)	compute_rms.R	Truncated
E	ACF Step/Stride Regularity	acf_gait_regularity.R	Truncated norm
F + G	AMI + Rosenstein + LDS/ACI (4 axes, parallel)	ami.R + rosenstein_divergence.R + fit_divergence_exponents.R	Resampled
H	[Optional] Export divergence curves	write.csv	Resampled
I	[Optional] Export resampled signals	write.csv	Resampled

3.1 Across-Axis Parallelization

The nonlinear analysis (steps F+G) is the computational bottleneck, accounting for approximately 90% of per-file processing time. Since the four axes (AP, V, ML, vector norm) are fully independent (each has its own AMI-determined delay, its own phase space reconstruction, and its own divergence curve) they can be processed simultaneously.
The batch script creates a PSOCK cluster at startup with up to 4 workers (one per axis) using the parallel package. The three analysis scripts (`ami.R`, `rosenstein_divergence.R`, `fit_divergence_exponents.R`) are sourced on each worker via `clusterEvalQ()`. Configuration parameters (constant across files) are exported once at startup via `clusterExport()`. For each file, the resampled signals are exported to workers before dispatching the four axes via `parLapply()`.

If cluster creation fails (e.g., on single-core systems or restricted environments), the script falls back transparently to sequential lapply() with identical results. The parallelization mode is reported in the startup message and in the batch summary.
The number of workers is controlled by the N_PARALLEL_WORKERS variable (default: 4, line 178 in `run_gait_analysis_batch.R`). Setting this to 1 forces sequential execution,

which can be useful for debugging or for environments where `parallel::makeCluster()` is not supported.

4. Function Reference

4.1 read_racing_config()

Reads all parameters from the 1_Configuration sheet of a RACING config Excel file, validates them, computes derived values, and returns a named list ready for batch use.

Parameters

Parameter	Type	Default	Description
xlsx_path	character	(required)	Path to the RACING configuration Excel file
verbose	logical	TRUE	Print a formatted summary of loaded parameters

Return Value

A named list containing all configuration parameters, type-converted and validated. The list includes:

Group	Parameters	Notes
Input Data	data_folder, sampling_freq, col_ap, col_v, col_ml, accel_units, csv_header, csv_delim	Read from rows 8-15
Preprocessing	apply_tilt, auto_orient	Read from rows 20-21
Signal Processing	target_steps, samples_per_step, target_samples	target_samples computed (not from Excel formula)
SF Detection	sf_freq_min, sf_freq_max	Read from rows 33-34
Linear Metrics	acf_max_lag, peak_min_distance	Read from rows 39-40
AMI / Phase Space	ami_n_bins, ami_n_lags, embedding_dim	Read from rows 45-47
Divergence	divergence_length, mean_period, lds_range_end, aci_range_start, aci_range_end	Read from rows 52-56
Output	output_folder, study_name, export_div_curves, export_resampled	Read from rows 60-63
Derived	acf_max_lag_sec, lds_steps, aci_stride_range, config_file	Computed from above

Usage Example

```
source("read_config.R")
config <- read_racing_config("RACING_config.xlsx")
config$sampling_freq   # 256
config$target_steps    # 250
config$target_samples  # 18750
```

4.2 validate_racing_config()

Internal validation function called by read_racing_config(). Checks for missing required parameters, invalid types, out-of-range values, and logical consistency. Returns a list with valid (logical), errors (character vector), and warnings (character vector).

Validation checks include: 9 required parameters (presence), data folder existence, column indices positive and distinct, acceleration units, delimiter sanity, SF frequency range, ACF

lag sanity, AMI bins minimum, ACI range consistency versus divergence curve length, LDS range positive, embedding dimension range (2-15), and mean_period versus samples_per_step consistency.

4.3 print_config_summary()

Displays a formatted ASCII box summarizing all configuration parameters grouped by category: Input Data, Preprocessing, Signal Processing, Nonlinear Analysis, and Output. Called automatically by read_racing_config() when verbose=TRUE.

4.4 import_csv_lowback()

Imports a single CSV file containing tri-axial accelerometer data using parameters from the RACING configuration. Designed for direct integration with the batch pipeline.

Parameters

Parameter	Type	Default	Description
file_path	character	(required)	Full path to the CSV file to import
config	list	(required)	Configuration from read_racing_config()
verbose	logical	FALSE	Print diagnostic messages during import

Return Value

A data.frame with 3 columns (ap, v, ml) in g units, with the following attributes attached:

Attribute	Type	Description
sampling_rate	numeric	Sampling frequency in Hz
units	character	Always 'g' (converted if needed)
file	character	Original filename (basename only)
n_samples	integer	Number of rows
import_warnings	character/NULL	Any warnings generated during import
col_indices	named integer	Original column indices (ap, v, ml)

Key Features

Comment line detection: Automatically detects and skips lines starting with '#' at the top of the file. This supports datasets like LTMM that have comment headers without requiring an extra configuration parameter.

Fast import: Uses data.table::fread() when available (fast, robust encoding handling). Falls back to base R read.csv() if data.table is not installed.

Unit conversion: If accel_units is 'm/s2', automatically converts to g by dividing by 9.81.

Validation: Checks file existence, readability, column count, numeric data types, and missing values. Reports NA counts per axis.

4.5 check_csv_structure()

Diagnostic utility for troubleshooting CSV import problems. Displays file info, raw line preview, import attempt results, and data quality checks. Not called during batch processing; intended for interactive use when a file fails to import.

Parameters

Parameter	Type	Default	Description
file_path	character	(required)	Path to the CSV file to examine
config	list	(required)	Configuration from read_racing_config()
n_preview	integer	5	Number of raw lines to display

4.6 write_racing_results()

Creates a formatted Excel workbook with four sheets (Metadata, Results, Metrics, Notes) containing the batch analysis results. Implements comprehensive safety features to prevent data loss.

Parameters

Parameter	Type	Default	Description
results_df	data.frame	(required)	Results with 21 columns (one row per file)
config	list	(required)	Configuration from read_racing_config()
output_path	character	(required)	Full path for the output .xlsx file
template_path	character	NULL	Optional path to template (for Metrics/Notes sheets)
verbose	logical	TRUE	Print confirmation messages

Output Workbook Structure

Sheet	Content	Formatting
Metadata	Project name, date, config parameters, pipeline description	Bold headers, blue styling
Results	One row per processed file, 21 metric columns	Frozen header, auto-filter, number formatting, conditional row highlighting (yellow=warning, pink=error)
Metrics	Column documentation: name, type, description, units	Reference for interpreting Results columns
Notes	Quality control guidance, metric interpretation, config provenance	Topic + note pairs

Safety Features

The function implements a multi-layer safety strategy against data loss:

1. Path resolution: All paths converted to absolute immediately, preventing the 'ghost file' problem where saveWorkbook() writes relative to a changed working directory.

2. Emergency RDS backup: The results data frame is saved as an .rds file BEFORE attempting Excel generation. This ensures data survives even if Excel creation crashes.

3. Post-save verification: After saving, the function checks both file.exists() and file.size() > 0. This catches the case where saveWorkbook() reports success but the file does not actually appear on disk (observed with OneDrive/network drives).

4. CSV fallback: If Excel save fails, results are automatically written as CSV. The user is given instructions to regenerate Excel from the backup later.

5. Writability test: Before saving, the function tests that the output directory is writable. If not, it falls back to the working directory with a warning.

4.7 recover_results()

Utility function for recovering results after a crash or failed save. Reads the automatic RDS backup created by write_racing_results() and regenerates the Excel workbook.

Parameters

Parameter	Type	Description
rds_path	character	Path to the _backup.rds file
config	list or character	Config list (if in memory) or path to config .xlsx file
xlsx_path	character	Output path for the regenerated Excel file

Usage Example

```
source("read_config.R")
source("write_results.R")
cfg <- read_racing_config("my_config.xlsx")
recover_results("results/MyStudy_results_backup.rds",
                cfg, "C:/Users/me/Desktop/recovered.xlsx")
```

5. Batch Script Usage

5.1 File Organization

All R scripts must be in the same directory. The recommended project layout is:

```
my_study/
  RACING_config.xlsx           # Your configuration
  run_gait_analysis_batch.R    # Main entry point
  read_config.R                # Config reader
  import_csv_lowback.R         # CSV import
  tilt_correction.R            # Moe-Nilssen algorithm
  step_frequency_detection.R   # FFT step detection
  resample_signal.R            # Signal resampling
  truncate_resample.R          # Combined preprocessing
  compute_rms.R                # RMS metrics
  acf_gait_regularity.R        # ACF regularity
  ami.R                        # Average Mutual Information
  rosenstein_divergence.R       # Divergence curves
  fit_divergence_exponents.R   # LDS & ACI fitting
  write_results.R              # Excel output writer
  data/                        # CSV accelerometer files
  results/                     # Output (created automatically)
```

5.2 Running the Batch

Method 1: Command line. Run from terminal:

```
Rscript run_gait_analysis_batch.R path/to/your_config.xlsx
```

Method 2: RStudio. Set the config path and source the script:

```
config_path <- "RACING_config.xlsx"
source("run_gait_analysis_batch.R")
```

Method 3: Override script directory. If scripts are in a different folder:

```
scripts_folder <- "C:/path/to/racing/scripts"
config_path <- "my_config.xlsx"
source("run_gait_analysis_batch.R")
```

5.3 Required R Packages

Package	Purpose	Required?
readxl	Read configuration from Excel (.xlsx)	Yes

openxlsx	Write results to formatted Excel	Yes
gsignal	Polyphase signal resampling (MATLAB-compatible)	Yes*
signal	Fallback resampling (if gsignal unavailable)	Fallback
RANN	Fast nearest-neighbor search (KD-tree, ~10x speedup)	Recommended
data.table	Fast CSV file reading	Recommended
parallel	Across-axis parallelization (AMI + Rosenstein)	Base R (no install needed)

** Either gsignal or signal is required; gsignal is strongly recommended for MATLAB compatibility.*

5.4 Checkpoint and Recovery System

The batch script implements incremental checkpoints to protect against mid-batch crashes:

Checkpoint interval: Every 50 files (configurable via CHECKPOINT_INTERVAL variable), the current results are saved as both CSV and RDS files in the output folder.

On successful completion: The checkpoint CSV is deleted; the RDS backup is retained. The final Excel workbook is verified on disk.

On crash or failure: The checkpoint files contain all results processed so far. The batch summary provides recovery instructions.

6. Configuration Reference

All parameters are read from the 1_Configuration sheet of the RACING Excel file. Values are in column C at the row numbers specified below.

Row	Parameter	Default	Description
8	data_folder	/path/to/data	Folder containing CSV accelerometer files
9	sampling_frequency	256	Accelerometer sampling rate (Hz)
10-12	column_ap / column_v / column_ml	1 / 2 / 3	Column indices for AP, Vertical, ML axes
13	acceleration_units	g	'g' or 'm/s2'
14	csv_has_header	TRUE	Whether CSV files have a header row
15	csv_delimiter	,	Column delimiter: comma, semicolon, or tab
20	apply_tilt_correction	TRUE	Apply Moe-Nilssen tilt correction
21	auto_detect_orientation	TRUE	Auto-detect inverted sensor mounting
26	target_steps	250	Number of steps to extract from each signal
27	samples_per_step	75	Samples per step after resampling
33-34	sf_freq_min / sf_freq_max	1.4 / 3.0	Step frequency detection range (Hz)
39	acf_max_lag	500	Maximum ACF lag (samples)
40	peak_min_distance	60	Minimum distance between ACF peaks (samples)
45	ami_n_bins	64	Number of bins for AMI histogram
46	ami_n_lags	100	Number of lags to compute for AMI

47	embedding_dimension	5	Phase space embedding dimension
52	divergence_length	1800	Maximum divergence curve length (samples)
53	mean_period	75	Mean period for Theiler window (samples)
54	lds_range_end	0.5	LDS fitting range: 0 to this value (strides)
55-56	aci_range_start / aci_range_end	5 / 12	ACI fitting range (strides)
60	output_folder	/path/to/output	Output directory (created if needed)
61	study_name	MyStudy	Prefix for output files
62	export_divergence_curves	FALSE	Export per-file divergence curves as CSV
63	export_resampled_signals	FALSE	Export resampled signals as CSV

7. Error Handling

7.1 Fault Tolerance

The batch pipeline is designed so that one failed file never stops the entire batch. Each file is wrapped in a tryCatch() block. On error, the results row is filled with NAs and the error message is recorded in the warnings column. Processing continues with the next file.

Within each file, individual metric computations (ACF regularity, AMI, divergence, fitting) each have their own tryCatch blocks. A failure in one metric (e.g., AMI fails to find a minimum) does not prevent the other metrics from being computed.

7.2 Warning Collection

Warnings are collected at every stage and concatenated into the warnings column with semicolons. Typical warnings include:

Warning Source	Example	Meaning
Import	NA values: AP=0, V=3, ML=0	Missing data in CSV file
Tilt correction	Tilt: inverted orientation corrected	Sensor was mounted upside-down
Tilt correction	Tilt: extreme angle 45.2 deg	Large sensor tilt (>30 degrees)
Signal length	Signal too short: 180 steps available (target: 250)	Recording shorter than expected
AMI	AMI_x: no minimum found, using tau=10	AMI curve has no clear minimum; fallback used
Divergence	Divergence_z: ...	Phase space computation failed for one axis
Fitting	Fit_norm: ...	LDS/ACI linear fit failed for one axis

7.3 Batch Summary

After processing all files, the batch script prints a summary showing: total files processed, successful count, count with warnings, failed count, total elapsed time, per-file average time, parallelization mode (ON with worker count, or OFF), and output file location with verification status.

8. Dependencies

8.1 Script Dependencies

The batch script sources 12 R scripts at startup. All must be present in the same directory:

Script	Layer	Functions Provided
`read_config.R`	Layer 3	`read_racing_config()`, `validate_racing_config()`, `print_config_summary()`
`import_csv_lowback.R`	Layer 3	`import_csv_lowback()`, `check_csv_structure()`
`tilt_correction.R`	Layer 1	`tilt_correction()`
`step_frequency_detection.R`	Layer 1	`step_frequency_fft()`
`resample_signal.R`	Layer 1	`resample_signal()`, `resample_log()`, `resample_log_header()`, `resample_log_sep()`
`truncate_resample.R`	Layer 2	`truncate_resample()`
`compute_rms.R`	Layer 1	`compute_rms()`
`acf_gait_regularity.R`	Layer 1	`compute_gait_regularity()`
`ami.R`	Layer 1	`ami()`
`rosenstein_divergence.R`	Layer 1	`rosenstein_divergence()`
`fit_divergence_exponents.R`	Layer 1	`fit_divergence_exponents()`
`write_results.R`	Layer 3	`write_racing_results()`, `recover_results()`

9. References

Moe-Nilssen, R. (1998). A new method for evaluating motor control in gait under real-life environmental conditions. Part 1: The instrument. Clinical Biomechanics, 13(4-5), 320-327.

Fraser, A. M., & Swinney, H. L. (1986). Independent coordinates for strange attractors from mutual information. Physical Review A, 33(2), 1134-1140.

Rosenstein, M. T., Collins, J. J., & De Luca, C. J. (1993). A practical method for calculating largest Lyapunov exponents from small data sets. Physica D, 65(1-2), 117-134.

Piergiovanni, S., & Terrier, P. (2024). Effects of metronome walking on long-term attractor divergence and correlation structure of gait. Scientific Reports, 14, 15784.

Piergiovanni, S., & Terrier, P. (2024). Validity of linear and nonlinear measures of gait variability to characterize aging gait with a single lower back accelerometer. Sensors, 24(23), 7427.

Appendix A: Quick Reference Card

Essential Constants

SAMPLING_FREQ = 256 Hz (Physilog sensor sampling rate). TARGET_STEPS = 250 (standardized step count, indoor). SAMPLES_PER_STEP = 75 (after resampling). TARGET_SAMPLES = 18,750 (250 x 75). EMBEDDING_DIM = 5 (phase space dimensions). LDS_RANGE = 0-75 samples (0-0.5 strides). ACI_RANGE = 750-1800 samples (5-12 strides). AMI_BINS = 64 (default histogram bins). AMI_LAGS = 100 (maximum search range). THEILER_WINDOW = 75 samples (temporal exclusion in NN search).

Pipeline Function Calls (Minimal)

```
# 1. Preprocessing
prep <- truncate_resample(acc_x, acc_y, acc_z)

# 2. Linear metrics (on truncated signals)
rms <- compute_rms(prep$truncated$z, prep$truncated$norm)
reg <- compute_gait_regularity(prep$truncated$norm)

# 3. Nonlinear metrics (on resampled signals, per axis)
ami_res <- ami(prep$resampled$norm, n_bins=64, n_lags=100)
div <- rosenstein_divergence(prep$resampled$norm,
         m=5, tau=ami_res$optimal_lag,
         mean_period=75, max_iter=1800)
exp <- fit_divergence_exponents(div$divergence)
```

Parallelization: 4 workers across axes (x, y, z, norm) for steps F+G. Controlled by N_PARALLEL_WORKERS in run_gait_analysis_batch.R. Set to 1 to force sequential mode.

Appendix B: Glossary

ACI: Attractor Complexity Index: long-term divergence slope (5-12 strides) measuring gait automaticity
ACF: Autocorrelation Function: measures signal self-similarity at different lags
ACIER: Attractor Complexity Index Empirical Rationalization: the source study (2021-2024)
AMI: Average Mutual Information: information-theoretic measure for optimal embedding delay
AP: Anteroposterior: the forward-backward axis of movement
DFA: Detrended Fluctuation Analysis: method for quantifying long-range correlations
Embedding dimension: Number of coordinates in reconstructed phase space (m = 5 in ACIER)
Fisher z-transform: atanh(r): variance-stabilizing transform for correlation coefficients
KD-tree: k-dimensional tree: data structure for efficient nearest-neighbor search
LDS: Local Dynamic Stability: short-term divergence rate (0-0.5 strides) measuring balance
LTMM: Long-Term Movement Monitoring: PhysioNet free-living accelerometer database
ML: Mediolateral: the side-to-side axis of movement
Phase space: Multidimensional space reconstructed from time-delayed copies of a signal
Polyphase filter: FIR filter implementation that combines upsampling, filtering, and downsampling
RACING: R-based Attractor Complexity INdex for Gait
RMS: Root Mean Square: average signal magnitude, proxy for walking speed
Rosenstein algorithm: Method for estimating largest Lyapunov exponent from small datasets (1993)
Stride: One complete gait cycle (two steps): left heel-strike to left heel-strike

Step: Half gait cycle: left heel-strike to right heel-strike (or vice versa)
Takens' theorem: Mathematical basis for reconstructing dynamical attractors from scalar time series
Tau (tau): Time delay for phase space embedding, determined by AMI first minimum
Theiler window: Temporal exclusion zone in nearest-neighbor search (prevents autocorrelation artifacts)
V: Vertical axis of movement

Appendix C: References

[1] Auvinet, B., Berrut, G., Touzard, C., et al. (2002). Reference data for normal subjects obtained with an accelerometric device. Gait & Posture, 16(2), 124-134.
[2] Dingwell, J. B., & Cusumano, J. P. (2000). Nonlinear time series analysis of normal and pathological human walking. Chaos, 10(4), 848-863.
[3] Fraser, A. M., & Swinney, H. L. (1986). Independent coordinates for strange attractors from mutual information. Physical Review A, 33(2), 1134-1140.
[4] Gasior, M., & Gonzalez, J. L. (2004). Improving FFT frequency measurement resolution by parabolic and Gaussian interpolation. AIP Conference Proceedings, 732(1), 276-285.
[5] Moe-Nilssen, R. (1998). A new method for evaluating motor control in gait under real-life environmental conditions. Part 1: The instrument. Clinical Biomechanics, 13(4-5), 320-327.
[6] Moe-Nilssen, R., & Helbostad, J. L. (2004). Estimation of gait cycle characteristics by trunk accelerometry. Journal of Biomechanics, 37(1), 121-126.
[7] Piergiovanni, S., & Terrier, P. (2024). Effects of metronome walking on long-term attractor divergence and correlation structure of gait. Scientific Reports, 14, 15784.
[8] Piergiovanni, S., & Terrier, P. (2024). Validity of linear and nonlinear measures of gait variability to characterize aging gait with a single lower back accelerometer. Sensors, 24(23), 7427.
[9] Rosenstein, M. T., Collins, J. J., & De Luca, C. J. (1993). A practical method for calculating largest Lyapunov exponents from small data sets. Physica D, 65(1-2), 117-134.
[10] Sekine, M., et al. (2013). A gait abnormality measure based on root mean square of trunk acceleration. Journal of NeuroEngineering and Rehabilitation, 10, 118.
[11] Takens, F. (1981). Detecting strange attractors in turbulence. In Dynamical Systems and Turbulence, Lecture Notes in Mathematics 898, 366-381. Springer.
[12] Terrier, P., & Reynard, F. (2018). Maximum Lyapunov exponent revisited: Long-term attractor divergence of gait dynamics is highly sensitive to the noise structure of stride intervals. Gait & Posture, 66, 236-241.
[13] Theiler, J. (1986). Spurious dimension from correlation algorithms applied to limited time-series data. Physical Review A, 34(3), 2427-2432.

RACING User Guide

Version 1.1 — March 2026

Open-source software developed under the ORD 2025 grant.
ACIER dataset: Zenodo DOI 10.5281/zenodo.10148824

HE-Arc Santé — HES-SO University of Applied Sciences and Arts Western Switzerland